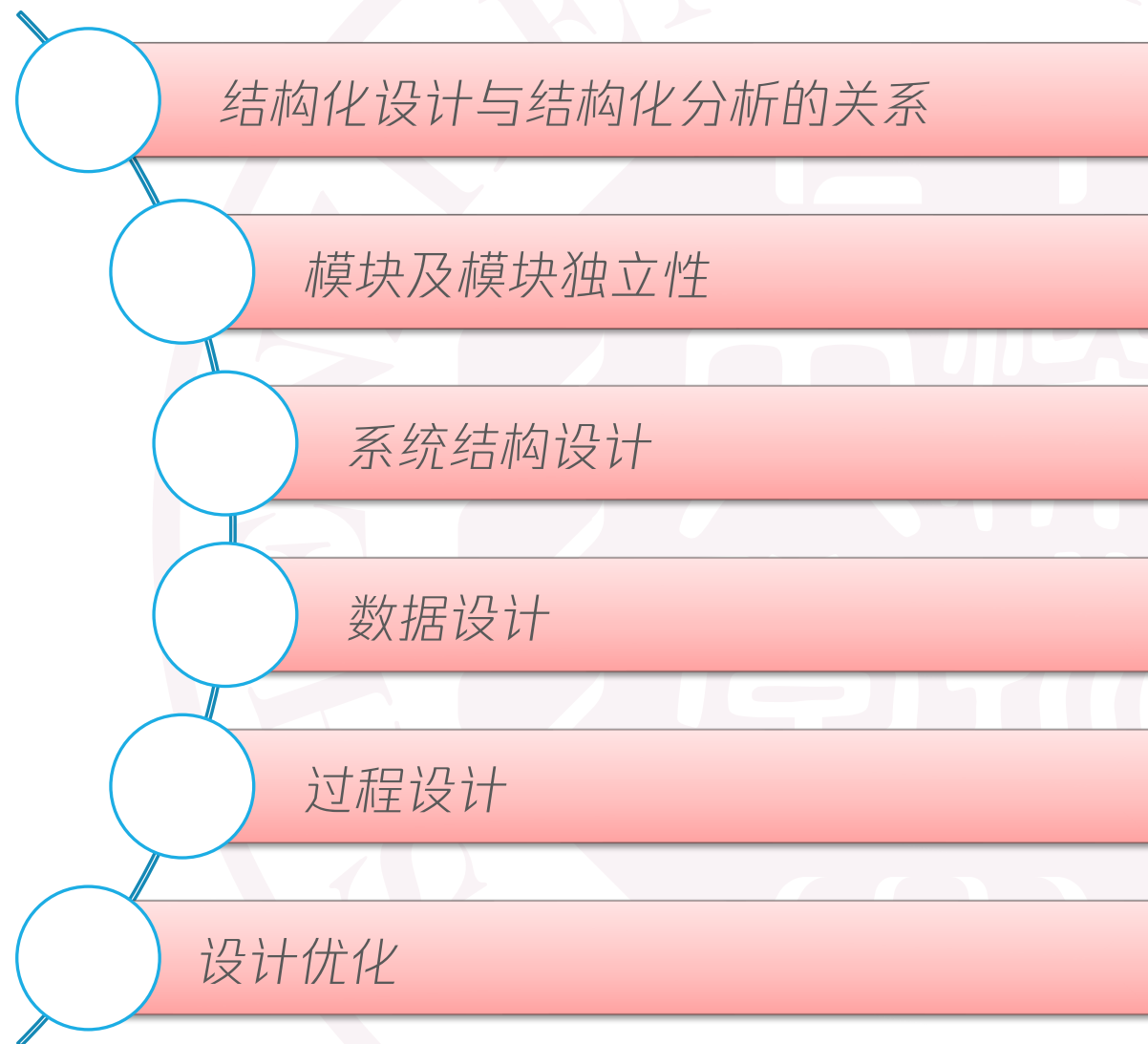




第4章 结构化设计

主要内容



概述

结构化设计阶段的主要任务是将需求分析阶段建模的数据处理需求转换为软件设计。

结构化设计涵盖内容

程序的处
理过程

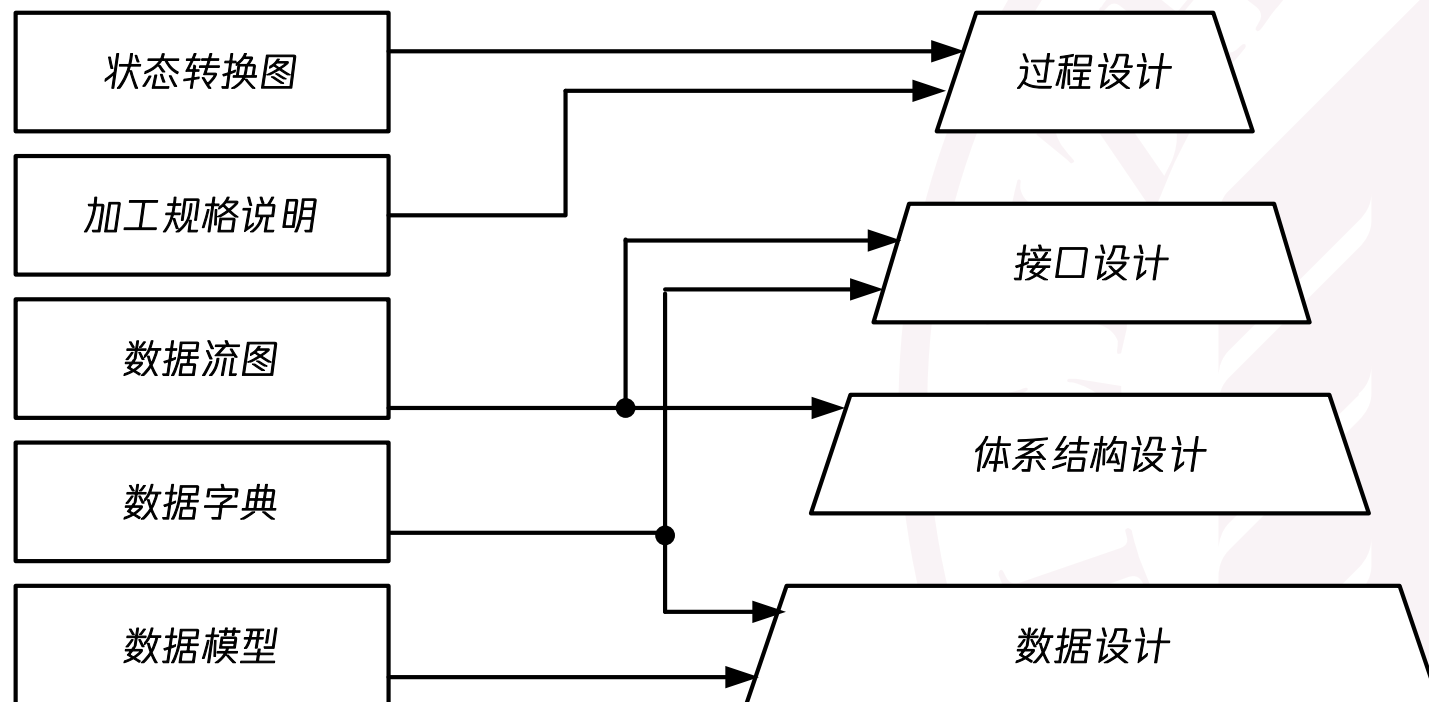
函数的内
部逻辑

函数的调
用关系

函数的接
口信息

待处理的
数据

1. 结构化设计与结构化分析的关系



2.模块化及耦合度评价

结构化程序设计语言中的模块

函数

宏定义

软件设计人员可以将需求分析中定义的处理映射为函数或者宏定义。

模块独立性评价指标：内聚、耦合

设计目标：高内聚，低耦合

2.模块化及耦合度评价

[1] 内聚

对**模块内部**各元素之间相关联程度的度量。

功能内聚

顺序内聚

信息内聚

通信内聚

过程内聚

时间内聚

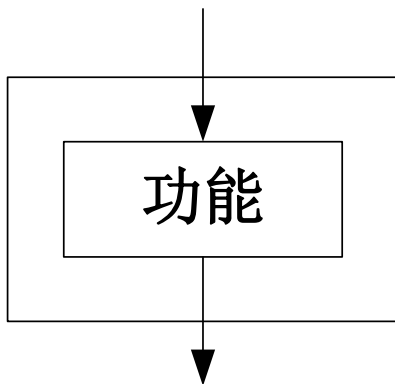
逻辑内聚

巧合内聚

2.模块化及耦合度评价

① 功能内聚

模块内部的所有元素均属于一个整体，共同完成模块的单一功能

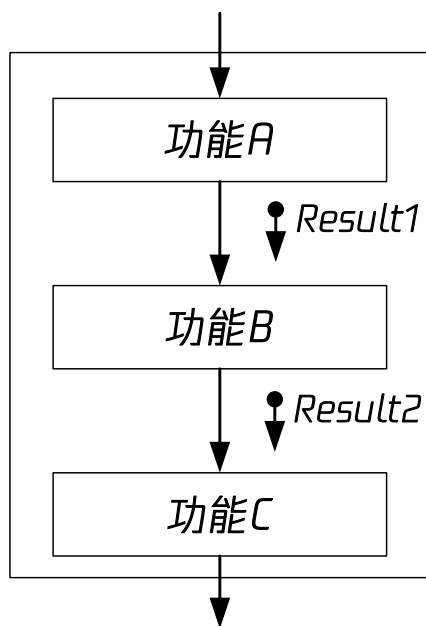


```
void FunctionConhesion()  
{  
    ....  
}
```

2.模块化及耦合度评价

② 顺序内聚

模块内封装了业务处理的多个功能步骤，上一个步骤的输出为下一个步骤的输入，依次执行。

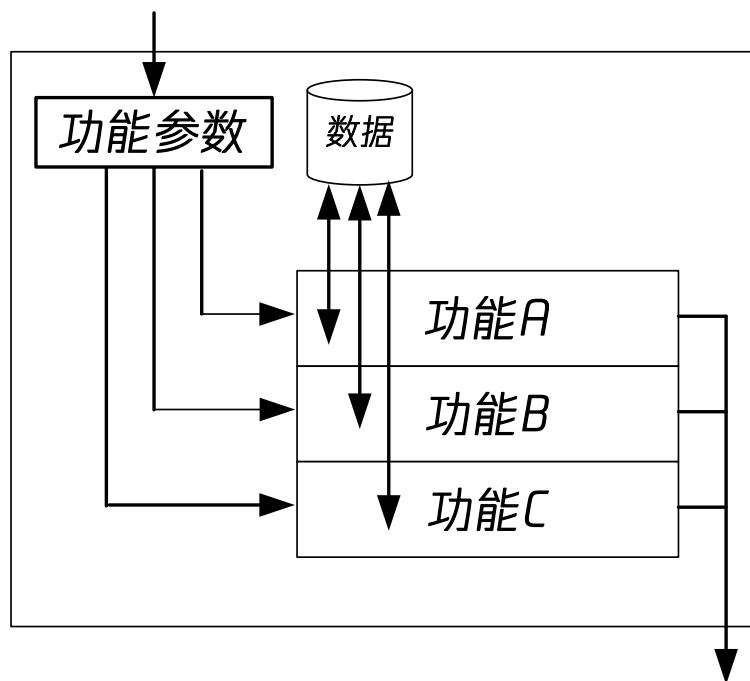


```
void SequenceCohesion()  
{  
    ...  
    x = FunctionA();  
    ...  
    y = FunctionB(x);  
    ...  
    z = FunctionC(y);  
}
```

2.模块化及耦合度评价

③ 信息内聚

多个共享同一个数据的功能封装在一起，模块根据不同的需求来执行特定功能，以达到信息隐藏的目的。



```
int InfoCohesion(int num, int OpType)
{
    // 共享数据
    int array[] = {1,2,3,4,5,6,7};

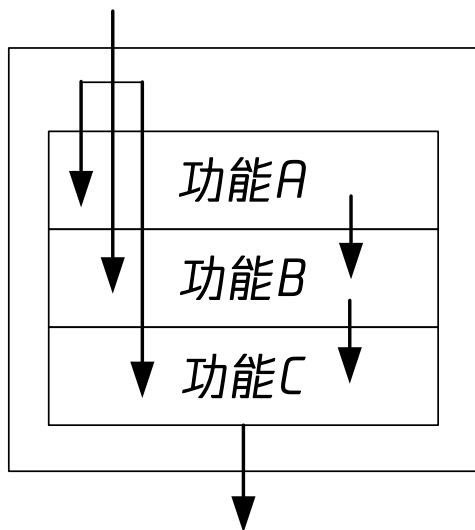
    if(1 == OpType)
    {
        FunctionA(array, num);
    }
    else if(2 == OpType)
    {
        FunctionB(array, num);
    }
    else if(3 == OpType)
    {
        FunctionC(array, num);
    }

    return 0;
}
```

2.模块化及耦合度评价

④ 通信内聚

模块内部封装了一系列与特定步骤顺序相关的元素，并且模块内的所有元素都使用同一个输入数据或产生同一个输出数据。

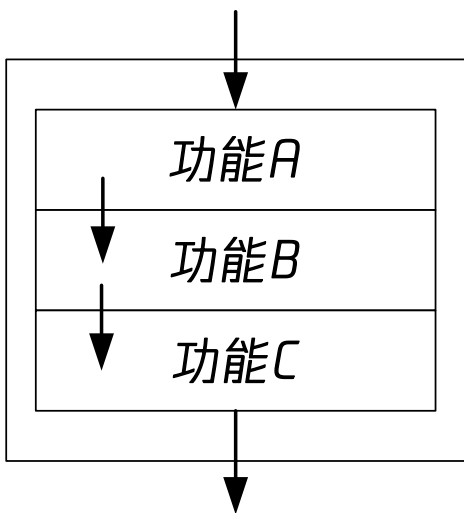


```
int CommunicationCohesion(int num)
{
    ...
    x = FunctionA(num);
    y = FunctionB(x, num);
    z = FunctionC(y, num);
    ...
    return 0;
}
```

2.模块化及耦合度评价

⑤ 过程内聚

模块内的处理元素是顺序相关的，即使相邻元素之间没有数据传递也必须以特定次序执行。

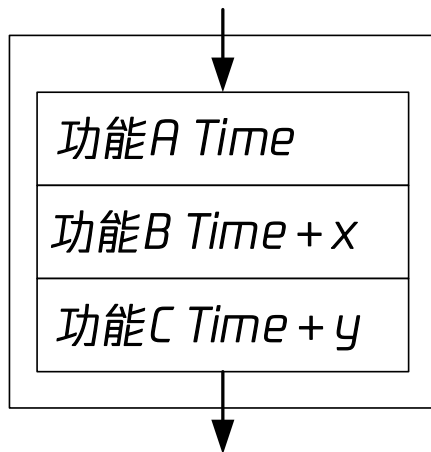


```
void ProceduralCohesion()  
{  
    ...  
    FunctionA();  
    ...  
    FunctionB();  
    ...  
    FunctionC();  
}
```

2.模块化及耦合度评价

⑥ 时间内聚

模块内包含的各个处理元素必须在同一段时间内执行，或者处理元素之间允许需要设置一定的时间限制。

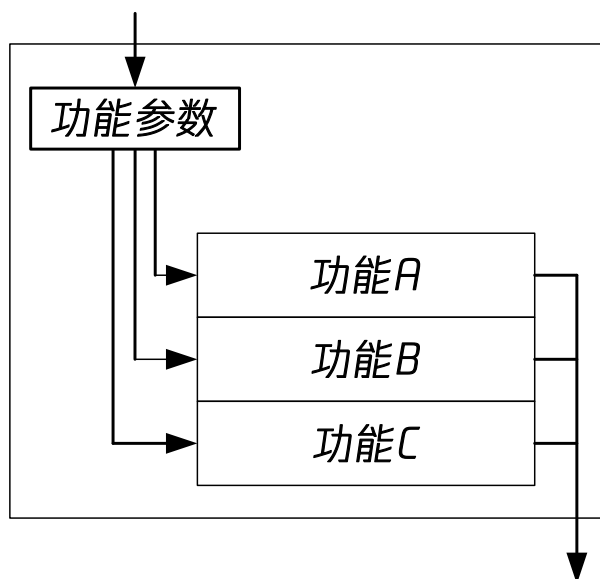


```
void TimeCohesion()  
{  
    FunctionA();  
    sleep(x);  
    FunctionB();  
    sleep(y);  
    FunctionC();  
}
```

2.模块化及耦合度评价

⑦ 逻辑内聚

多个相关的功能被集成到一个模块中，且每个功能的执行由传入模块的参数来决定。



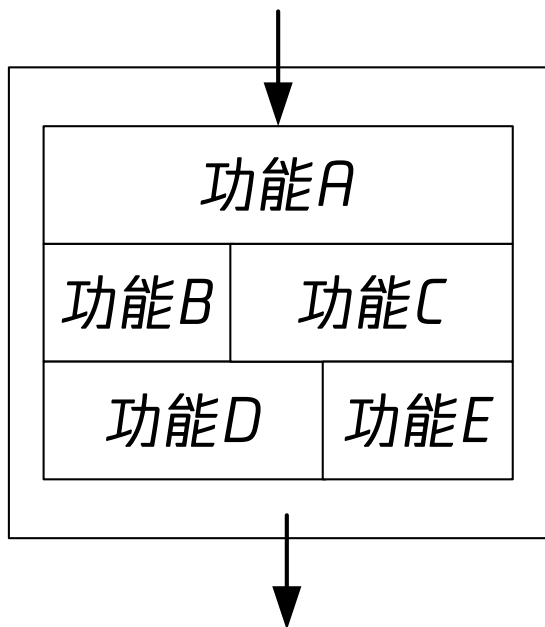
```
int LogicCohesion(int OpType)
{
    switch(OpType)
    {
        case 1: FunctionA();break;
        case 2: FunctionB();break;
        case 3: FunctionC();break;
        default: FunctionD();break;
    }

    return 0;
}
```

2.模块化及耦合度评价

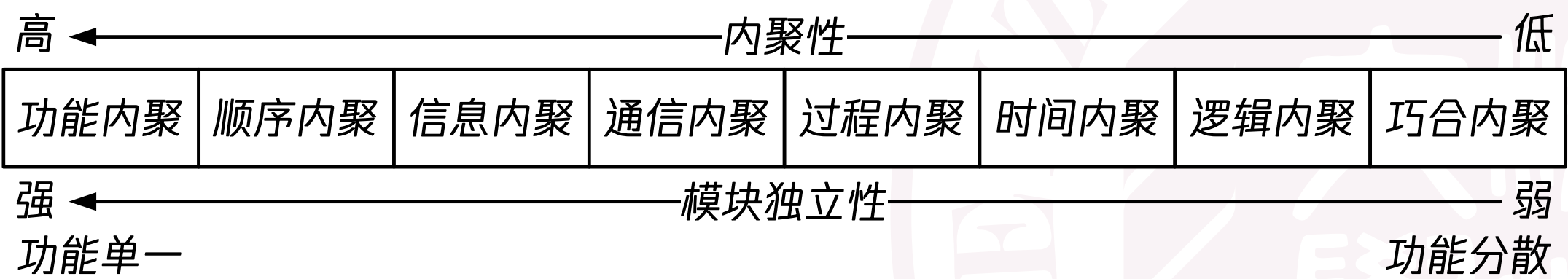
⑧ 巧合内聚

模块内的各个元素没有必要联系，或者即使有联系，也是很松散的联系。



2.模块化及耦合度评价

模块设计目标: 高内聚, 低耦合



2.模块化及耦合度评价

[2] 耦合

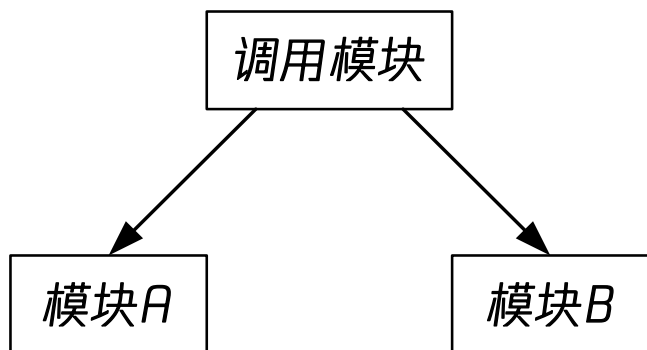
对模块之间联系的紧密程度进行度量。



2.模块化及耦合度评价

① 非直接耦合

模块之间没有直接关系，它们之间的联系完全是通过主模块或者调用模块来实现的。

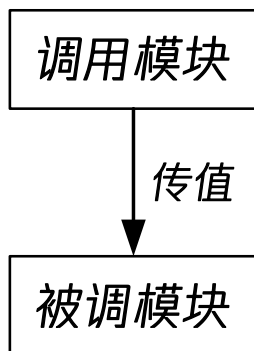


```
void SubFunctionA()  
{  
  
}  
  
void SubFunctionB()  
{  
  
}  
  
void IndirectCoupling()  
{  
    SubFunctionA();  
    SubFunctionB();  
}
```

2.模块化及耦合度评价

② 数据耦合

模块之间存在参数调用关系，且模块之间仅通过传值方式来交换信息。



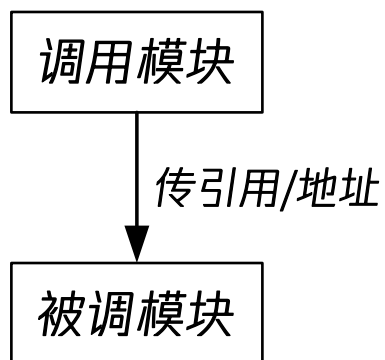
```
int SubFunction(int x, int y)
{
    return x > y ? x : y;
}

void DataCoupling()
{
    int temp = SubFunction(5, 6);
}
```

2.模块化及耦合度评价

③ 标记耦合

模块之间存在调用关系，且调用过程中使用引用、地址等方式来进行单向或者双向值传递。



```
void SubFunction(int &x, int &y)
{
    ...
}

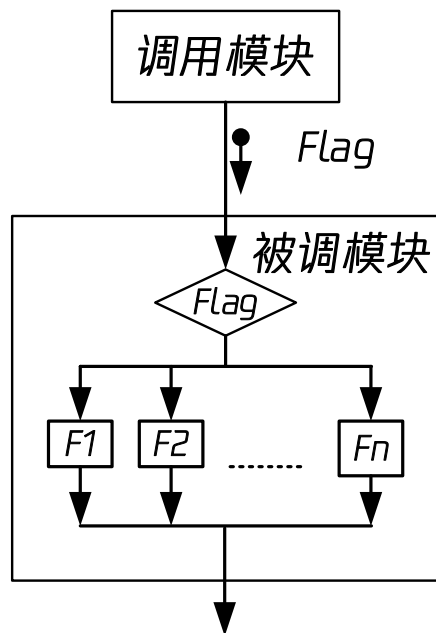
void MarkCoupling()
{
    int a = 10;
    int b = 20;

    SubFunction(a,b);
}
```

2.模块化及耦合度评价

④ 控制耦合

模块之间传递的信息中存在控制信息，调用模块通过参数、开关来控制被调模块的内部运行。



```
void SubFunction(int OpType)
{
    switch(OpType)
    {
        case 1: Function1();break;
        case 2: Function2();break;
        case 3: Function3();break;
        default: Function4();break;
    }
}

void ControlCoupling()
{
    SubFunction(1);
    SubFunction(2);
}
```

2.模块化及耦合度评价

⑤ 外部耦合

多个模块通过全局变量进行数据交换和共享。



```
int CommonPara = 0;

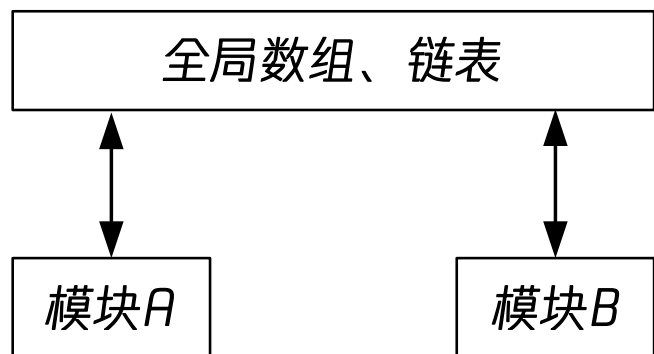
void FunctionA()
{
    // 使用, 修改 CommonPara
}

void FunctionB()
{
    // 使用, 修改 CommonPara
}
```

2.模块化及耦合度评价

⑥ 公共耦合

多个模块之间共享的全局数据为复杂数据类型，如链表，内存块等。

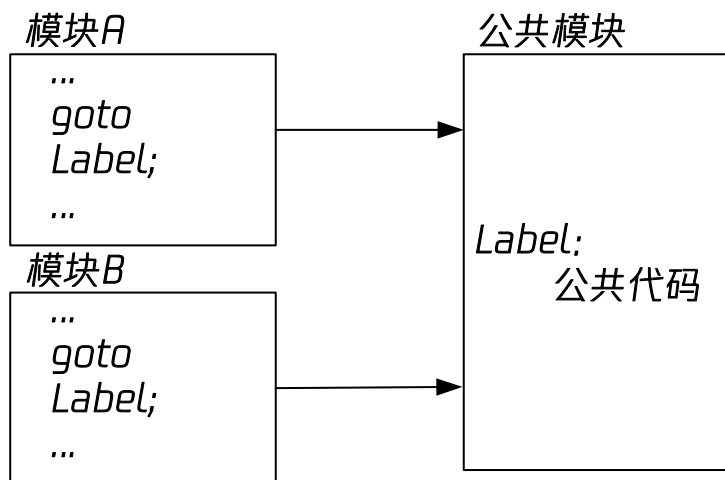


```
int CommonMemory[100];  
  
void FunctionA()  
{  
    //使用, 修改 CommonMemory  
}  
  
void FunctionB()  
{  
    //使用, 修改 CommonMemory  
}
```

2.模块化及耦合度评价

⑦ 内容耦合

模块通过特殊手段直接访问另一个模块的内部数据，或者模块通过非正常入口转到另一个模块内部，以及被调模块通过加注标记方式提供多个入口。



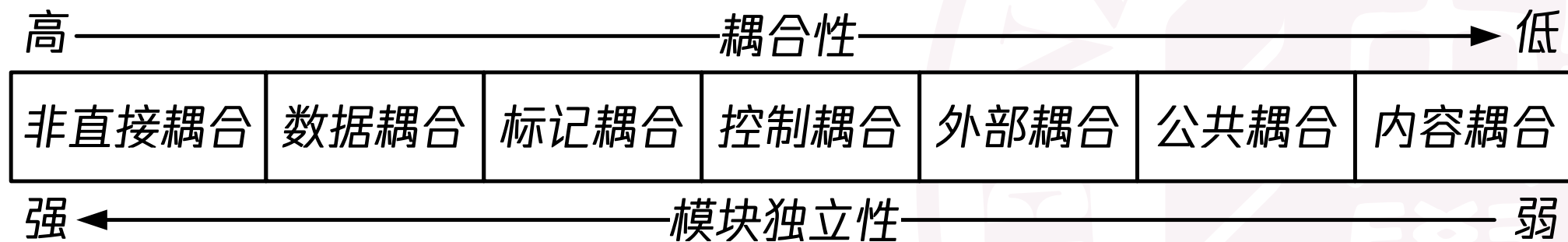
```
void CommonFunction()
{
    Label:
    // 公共程序代码
}

void FunctionA()
{
    goto Label;
}

void FunctionB()
{
    goto Label;
}
```

2.模块化及耦合度评价

模块设计目标：高内聚，**低耦合**



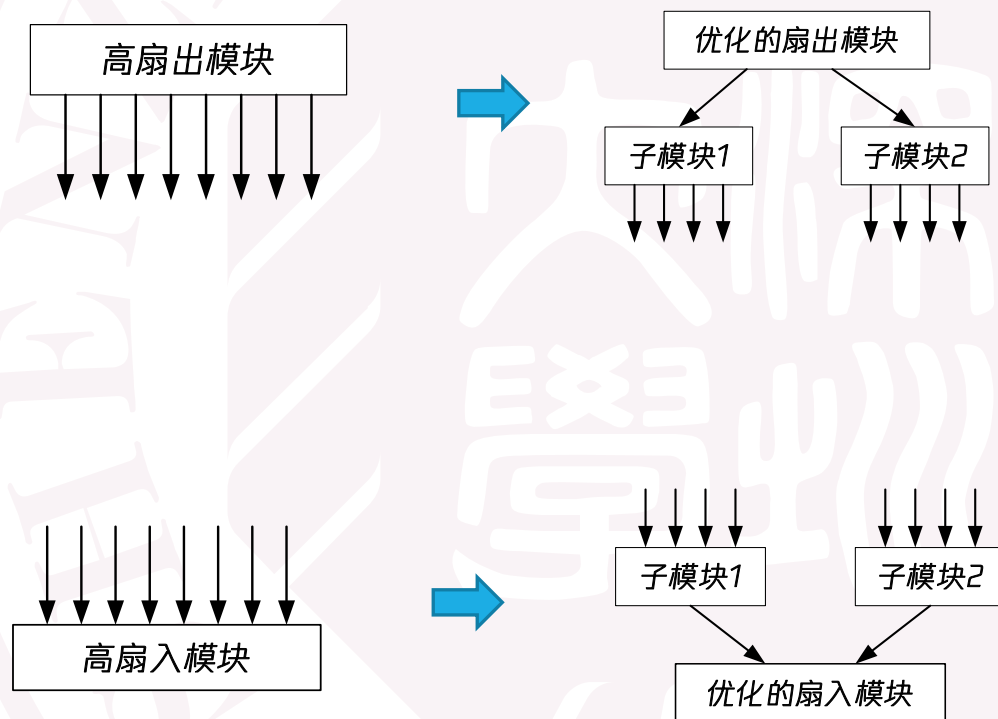
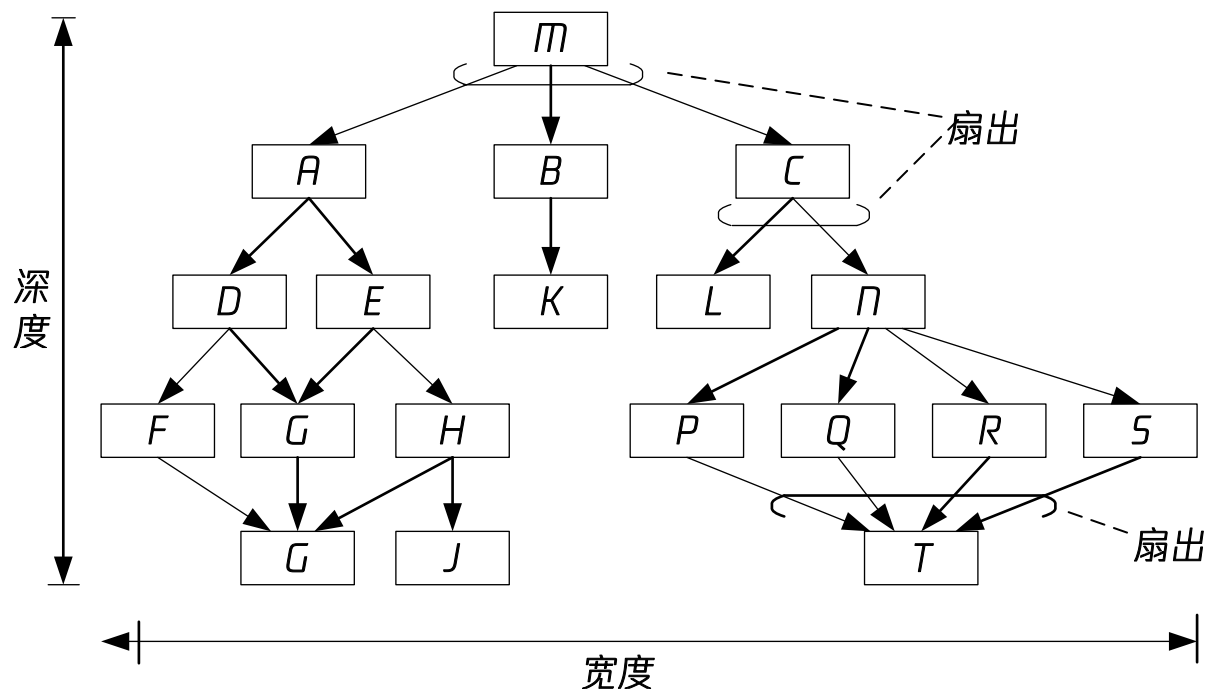
2.模块化及耦合度评价

[3] 启发式规则

- ✓ 改进软件结构提高模块独立性
- ✓ 模块的规模应该适中
- ✓ 软件架构的深度、宽度、扇入、扇出应当适中
- ✓ 模块的作用域应该在控制域之内
- ✓ 模块的功能应该可以预测

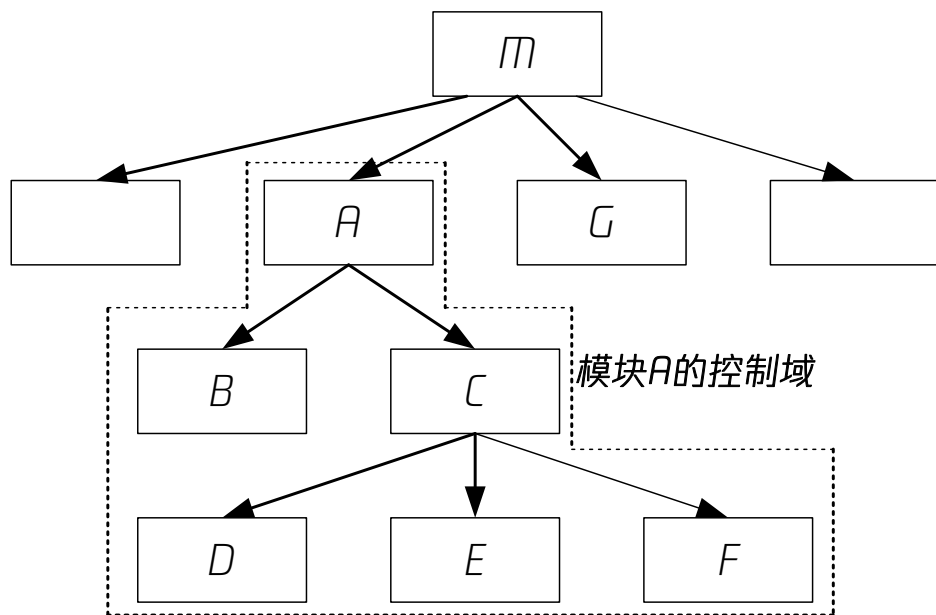
2.模块化及耦合度评价

软件架构的深度、宽度、扇入、扇出应当适中



2.模块化及耦合度评价

模块的作用域应该在控制域之内



简单:

模块B在模块A的控制域内, 模块A可以控制模块B的运行。

复杂:

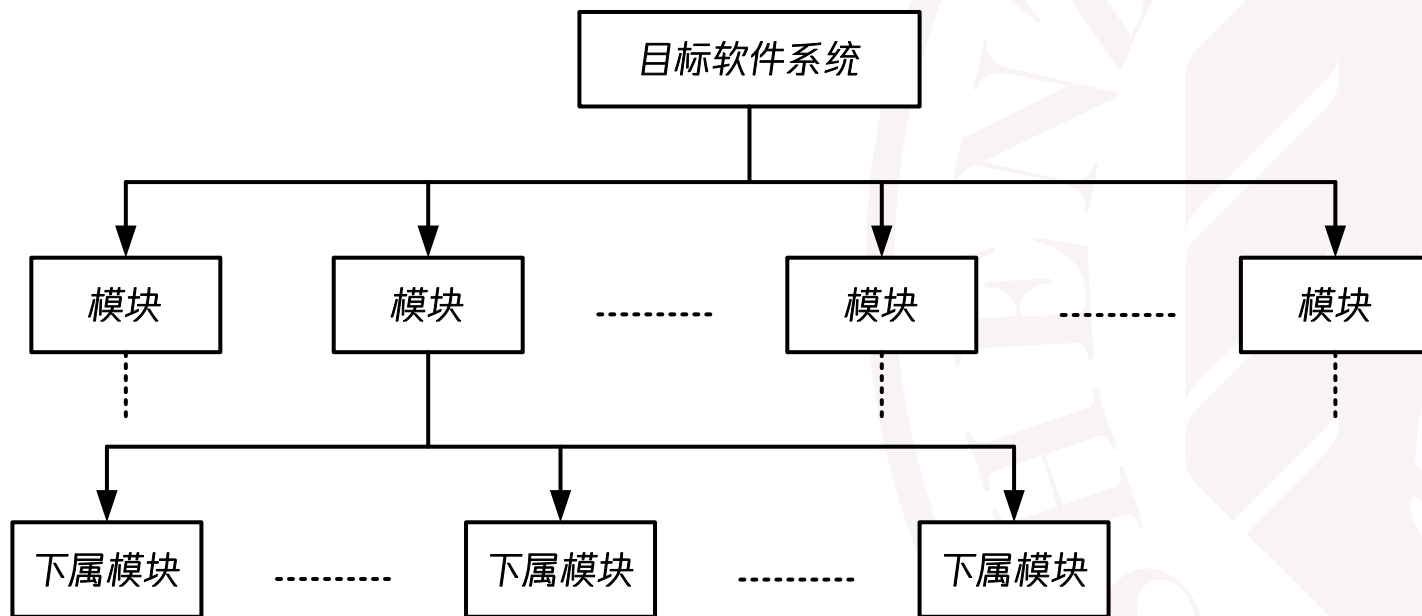
模块G不在模块A的控制域内。

如果模块A能够控制模块G的运行, 则该软件结构是不简单的。

3.系统结构设计

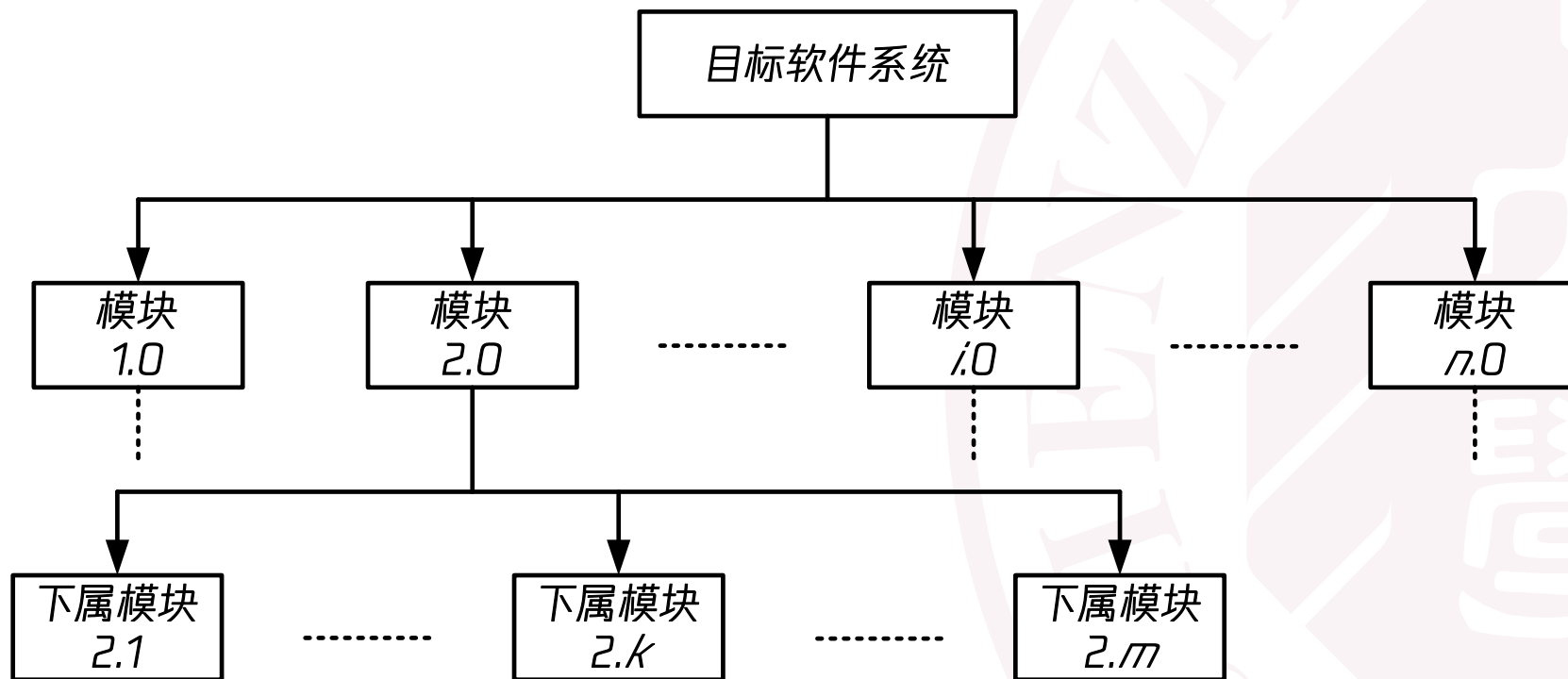
[1] 系统结构图

系统结构图，又称为层次图，是软件系统结构整体设计的图形化显示。



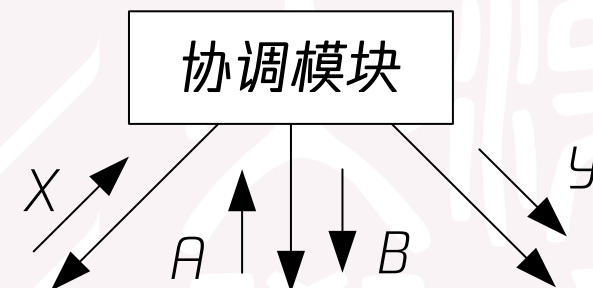
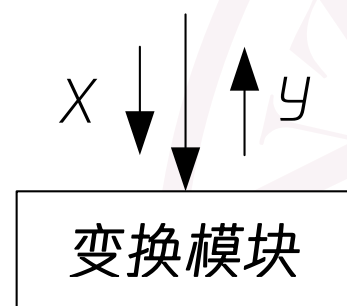
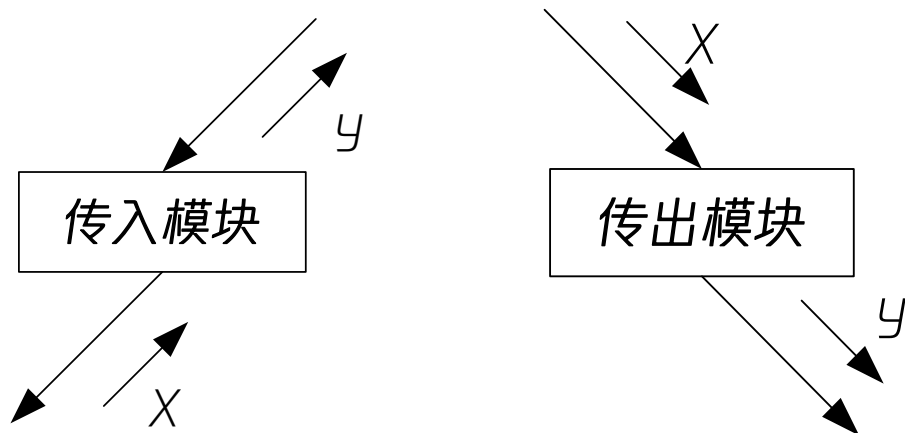
3.系统结构设计

为各个模块增加编号，便于跟踪各个模块的调用关系。



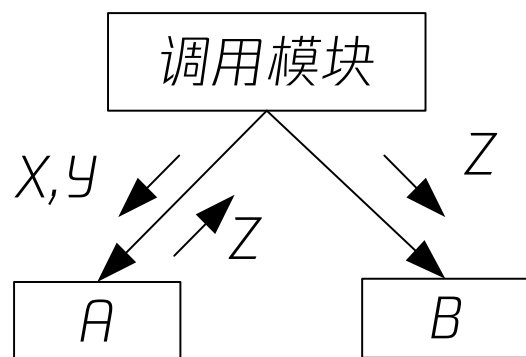
3.系统结构设计

加入数据流向的系统结构图

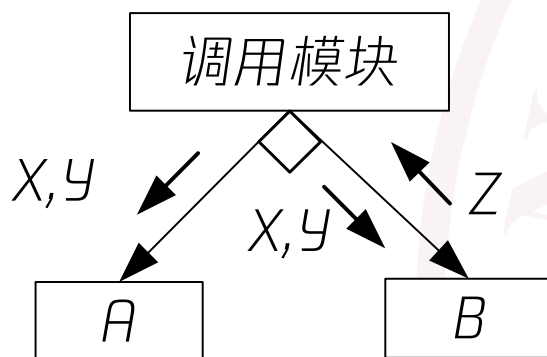


3.系统结构设计

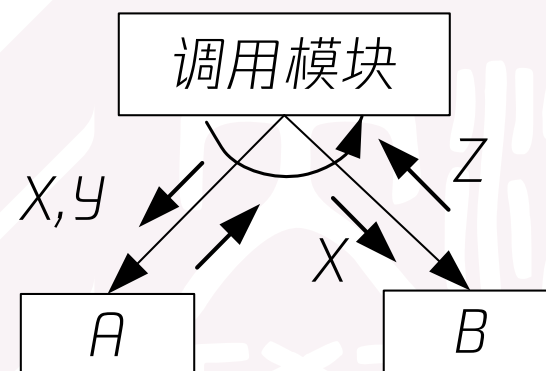
加入模块调用方式的系统结构图



简单调用



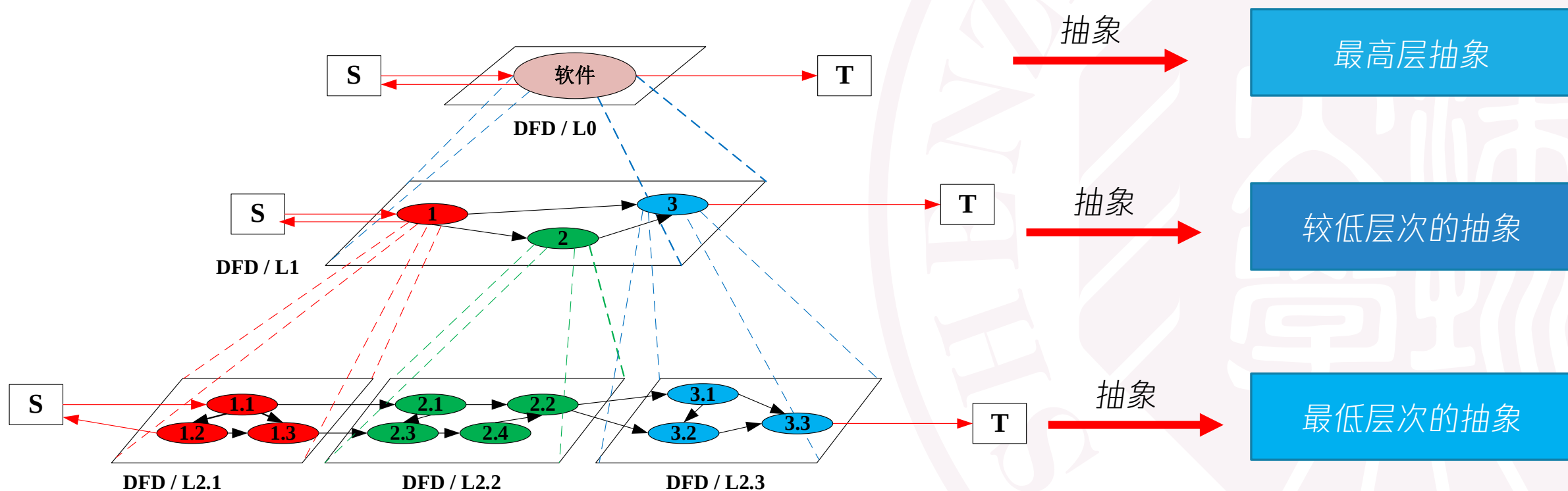
选择调用



循环调用

3.系统结构设计

采用分层细化方式进行建模，“自定向下、逐步求精”，有利于软件设计人员分解软件系统的功能内容。



3.系统结构设计

[2] HIPO图

层次图加输入/处理/输出图(Hierarchy Plus Input/Processing/Output)



3.系统结构设计

IPO表

系统: _____

作者: _____

模块: _____

日期: _____

编号: _____

被调用:

调用:

输入:

输出:

处理:

局部数据元素:

注释:

系统			
模块		编号	
作者		日期	
被调用		调用	
输入		输出	
处理			
局部数据		注释	

3.系统结构设计

(3) 面向数据流的设计方法

需求分析阶段得到的数据流图可以映射为系统结构。



3.系统结构设计

① 变换流

将“外部世界”输入的信息进行处理，然后再将处理后的信息输出到“外部世界”。

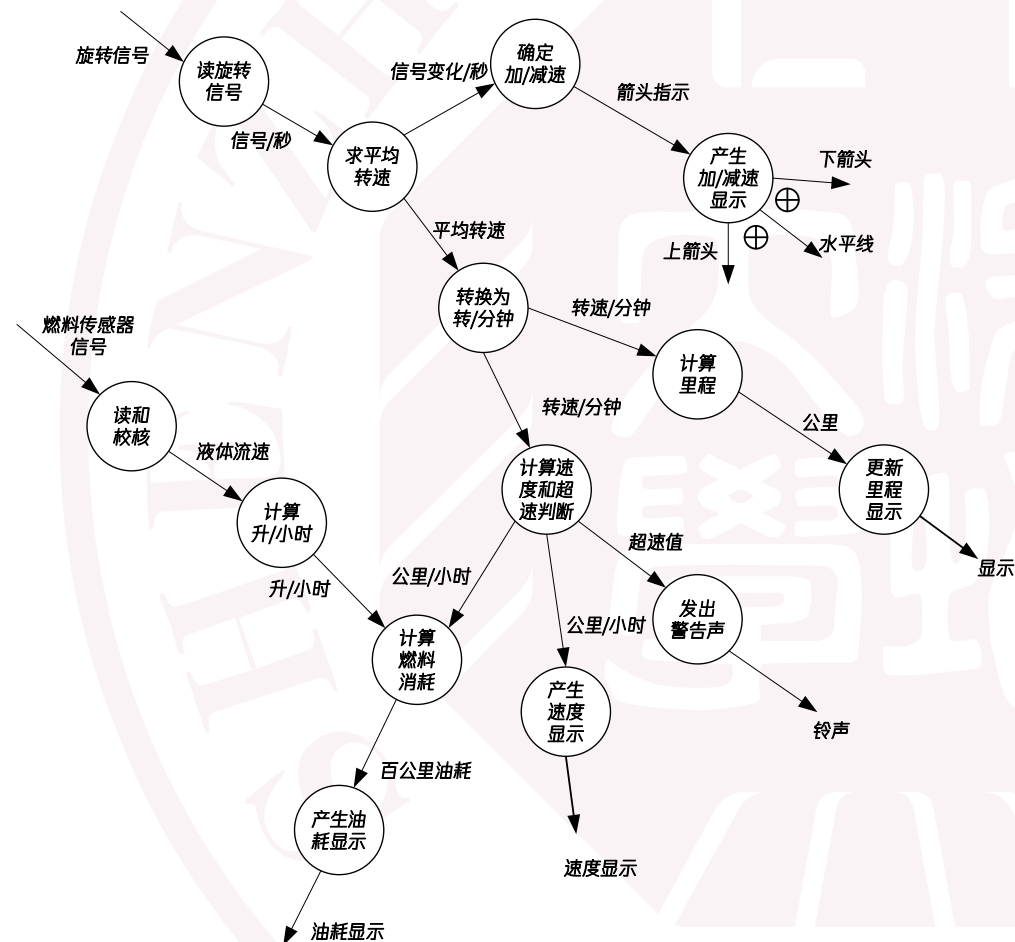


3.系统结构设计

变换流映射步骤:

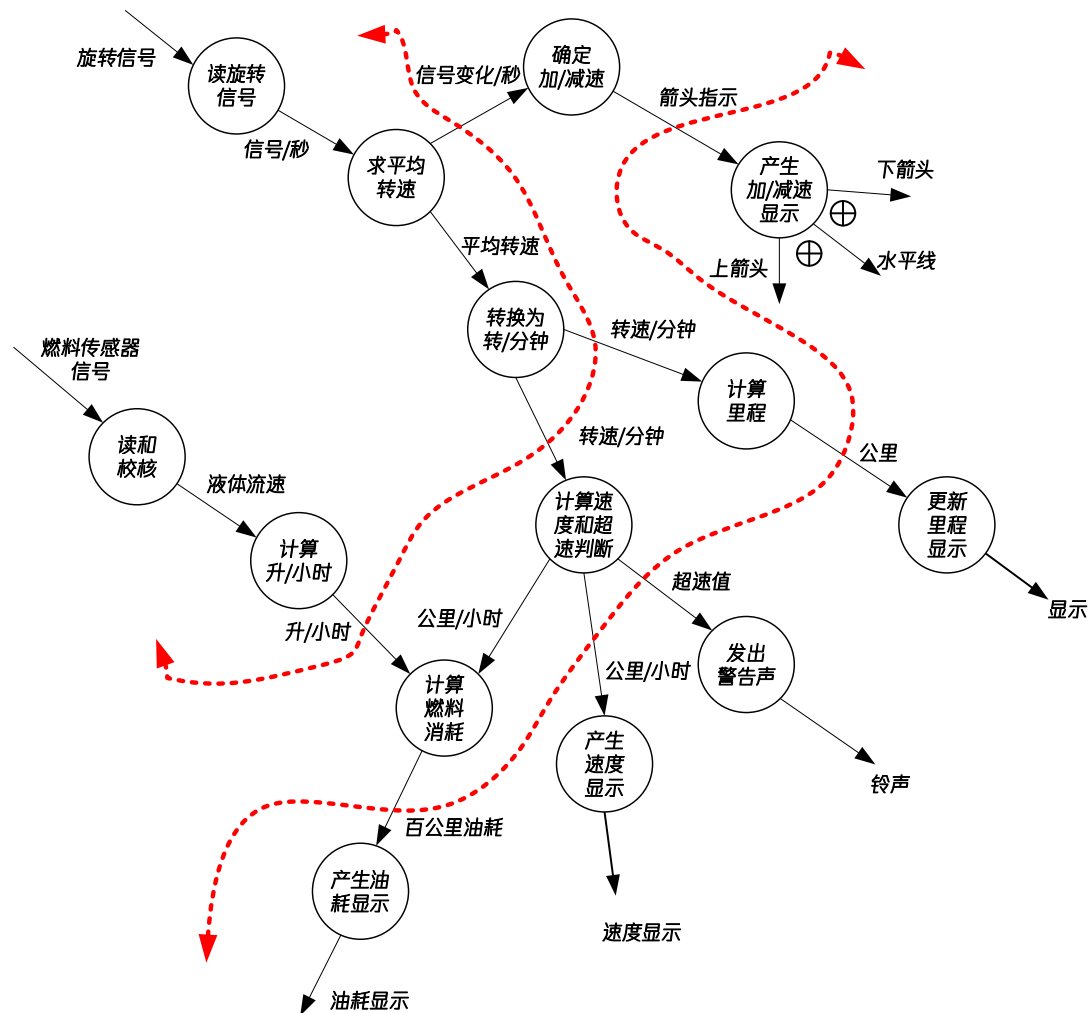
第一, 复查并精化数据流图

[汽车仪表盘控制系统案例]



3.系统结构设计

第二，确定输入流和输出流的边界，孤立出变换中心



输入数据部分包括:

“读旋转信号”、“求平均转速”、“转换为转/分钟”、“读和校核”和“计算升/小时”5个处理;

变换数据部分包括:

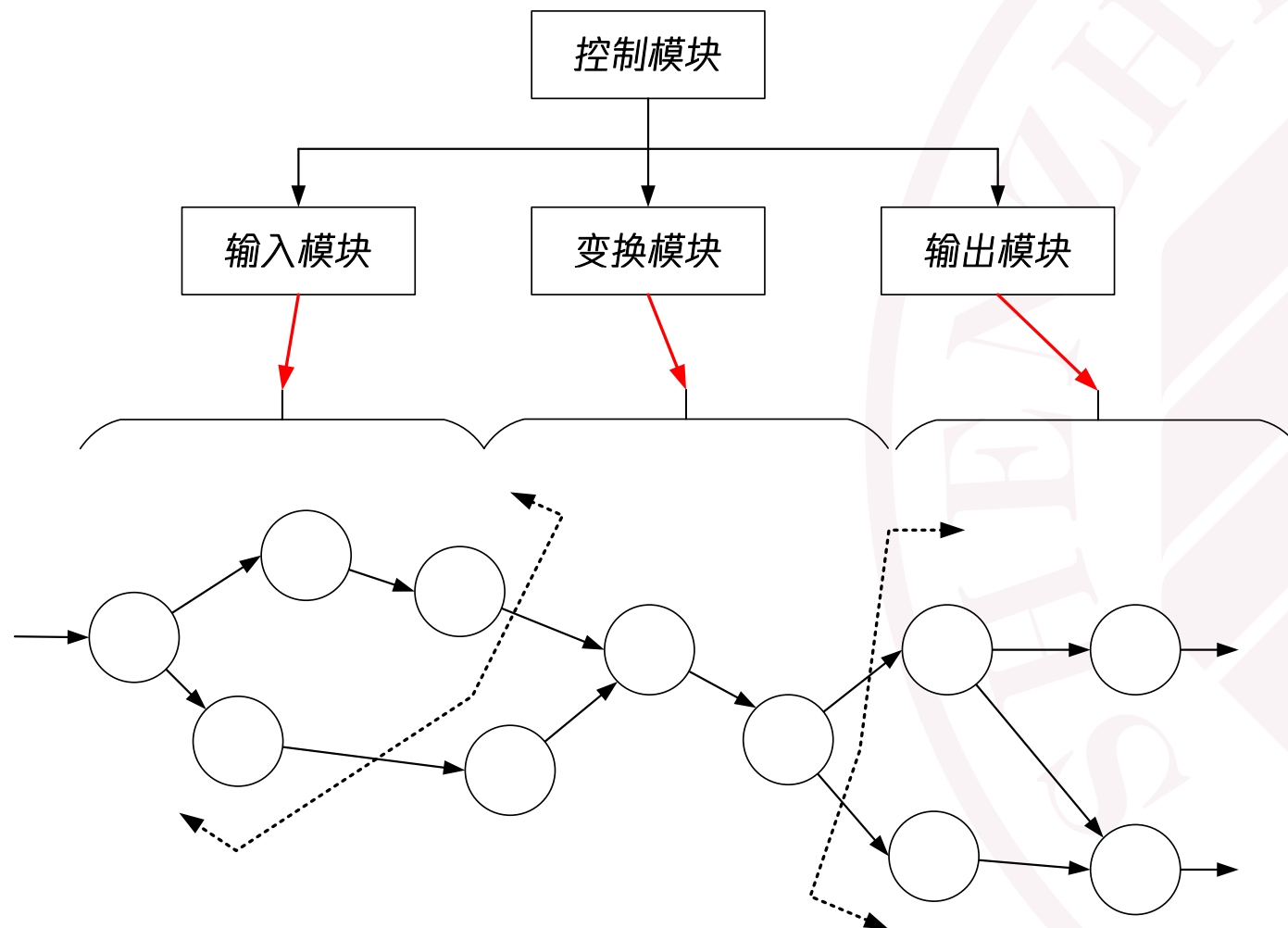
“确定加/减速”、“计算里程”、“计算速度和超速判断”和“计算燃料消耗”4个处理;

输出数据部分包括:

“产生加/减速显示”、“更新里程显示”、“发出警告声”、“产生速度显示”和“产生油耗显示”5个处理。

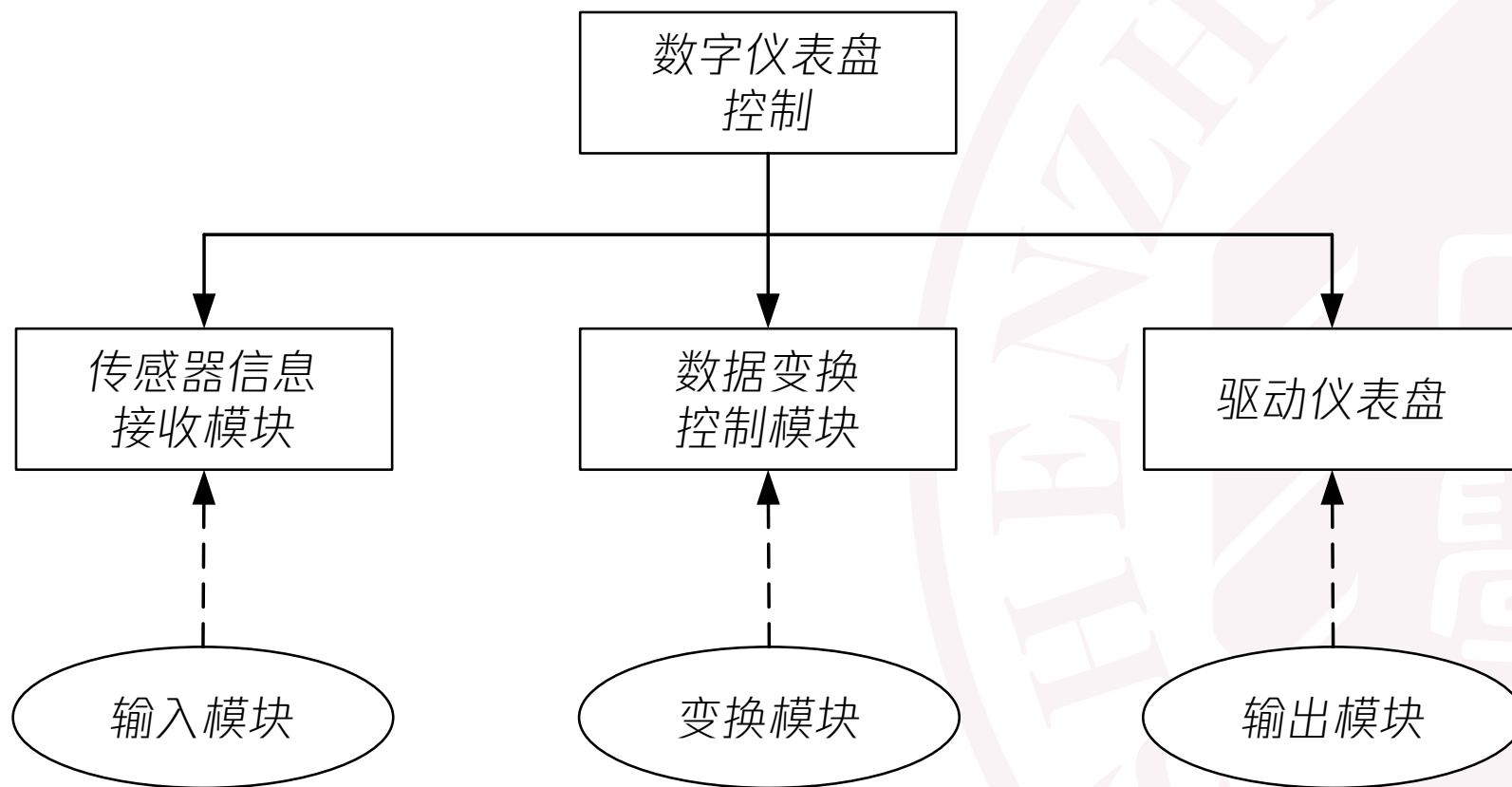
3.系统结构设计

第三，进行第一级分解，设计顶层和第一层模块



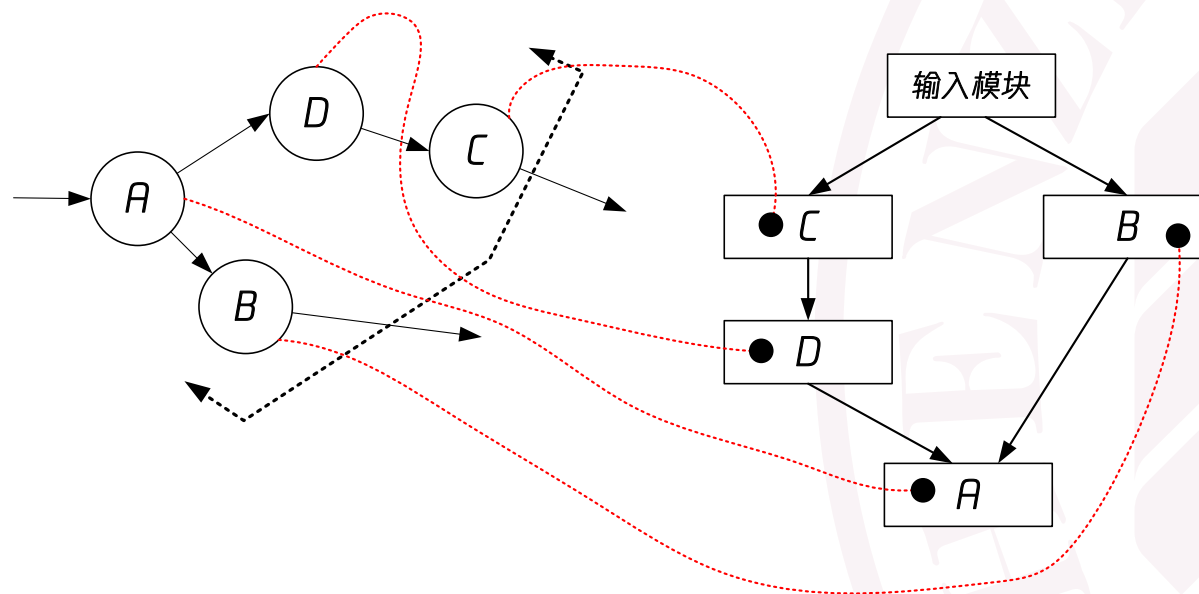
3.系统结构设计

[汽车仪表盘控制系统案例]



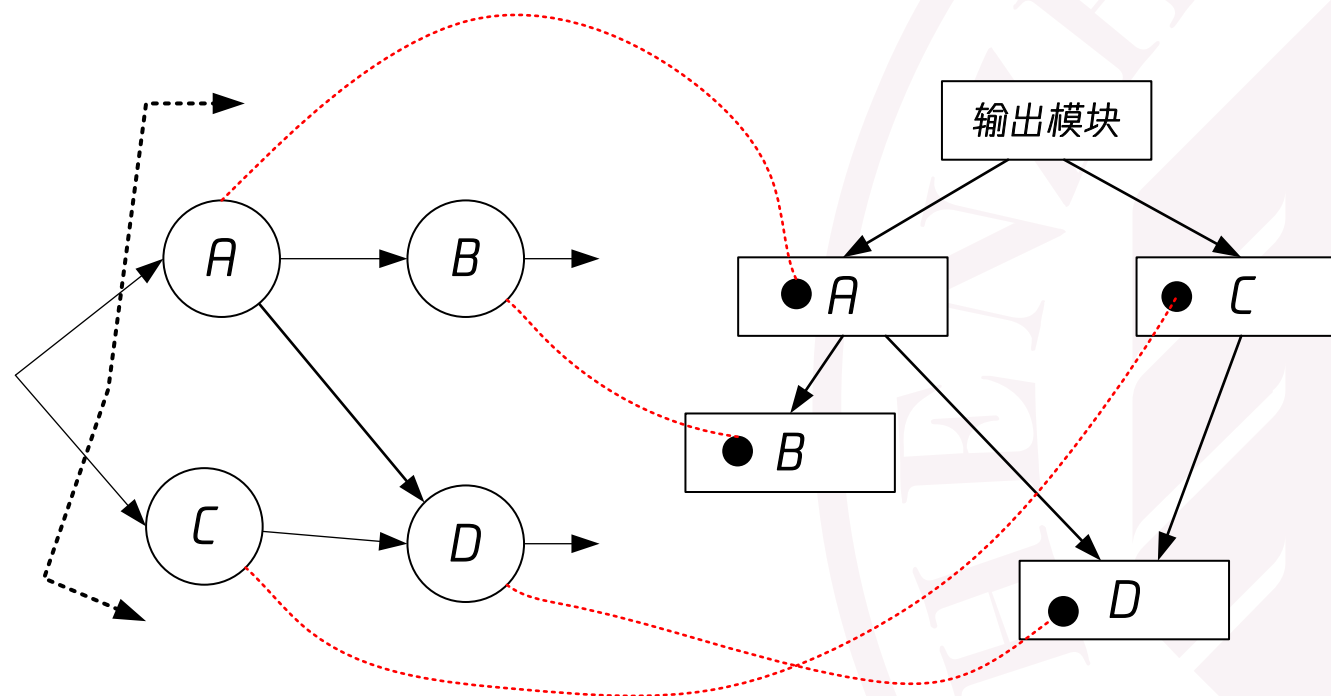
3.系统结构设计

第四，进行第二级分解，设计中、下层模块



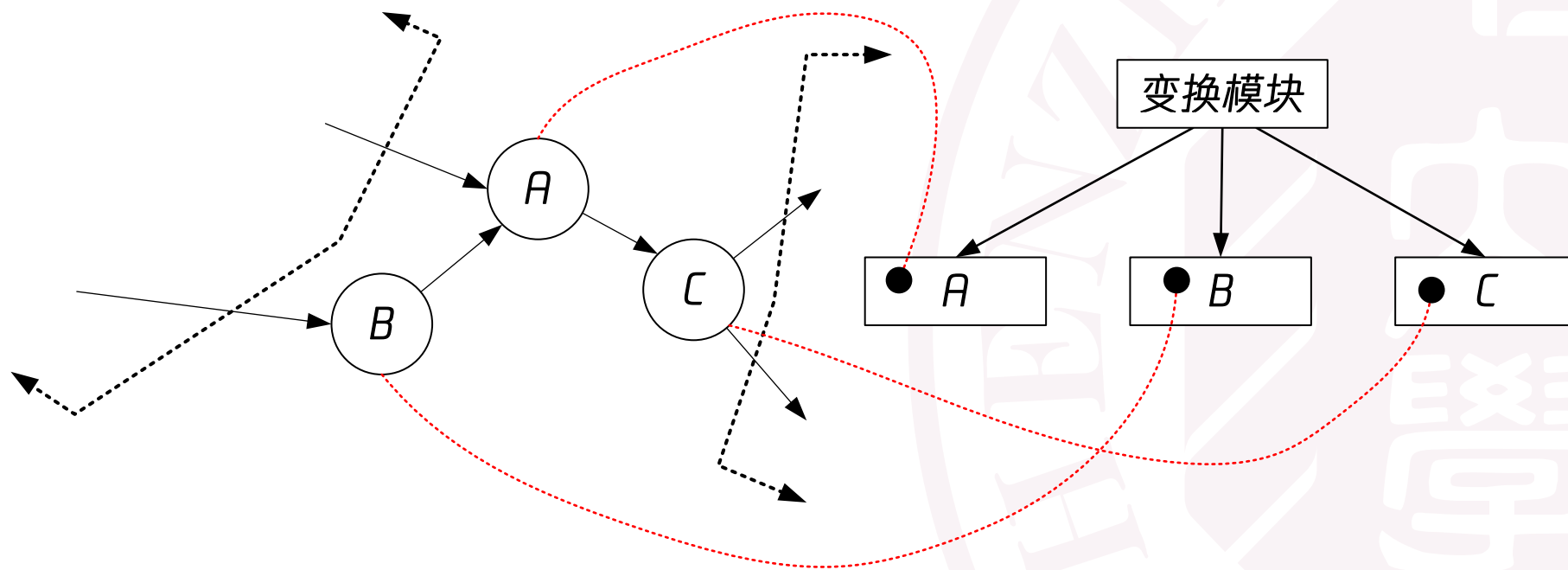
3.系统结构设计

第四，进行第二级分解，设计中、下层模块



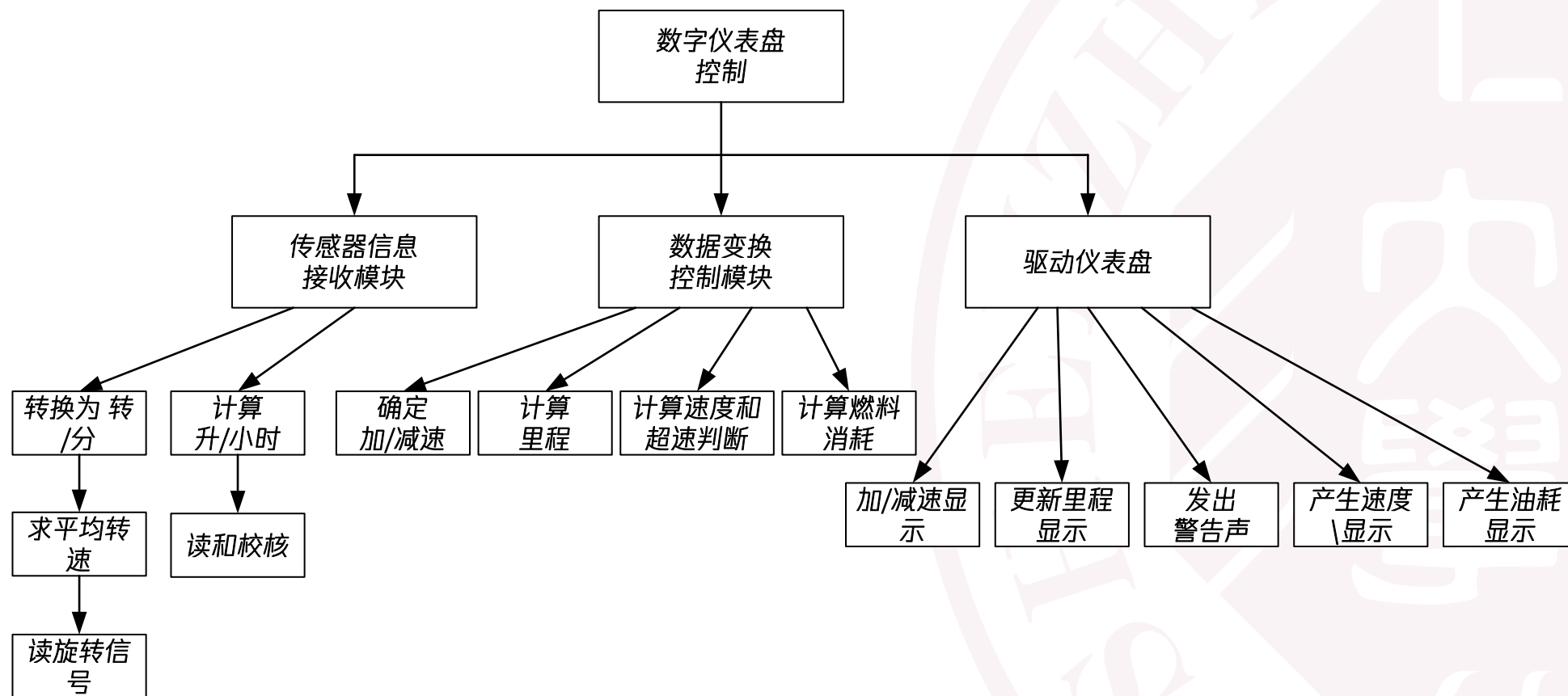
3.系统结构设计

第四，进行第二级分解，设计中、下层模块



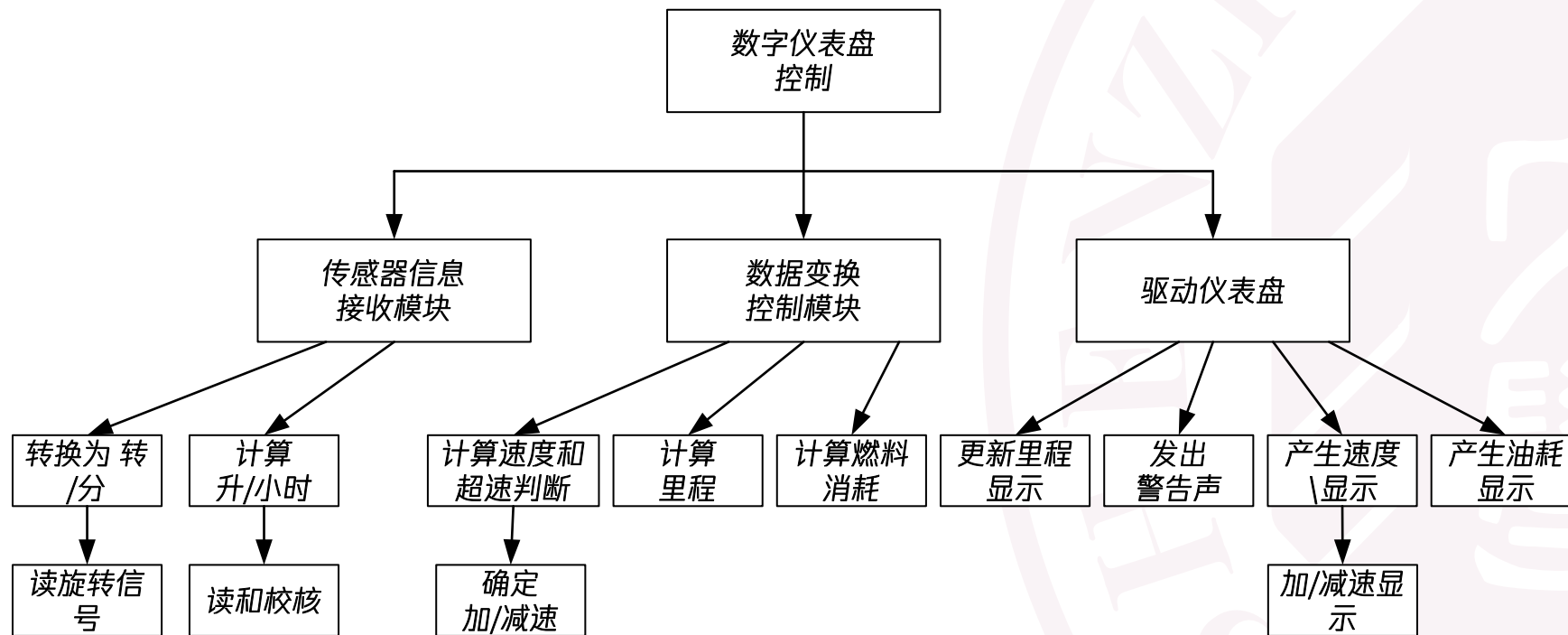
3.系统结构设计

[汽车仪表盘控制系统案例]



3.系统结构设计

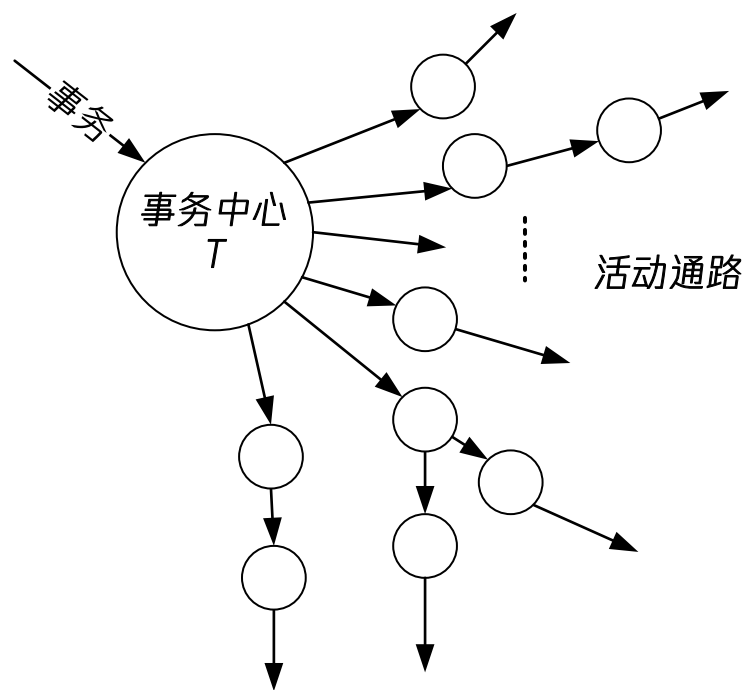
第五，软件层次结构精化



3.系统结构设计

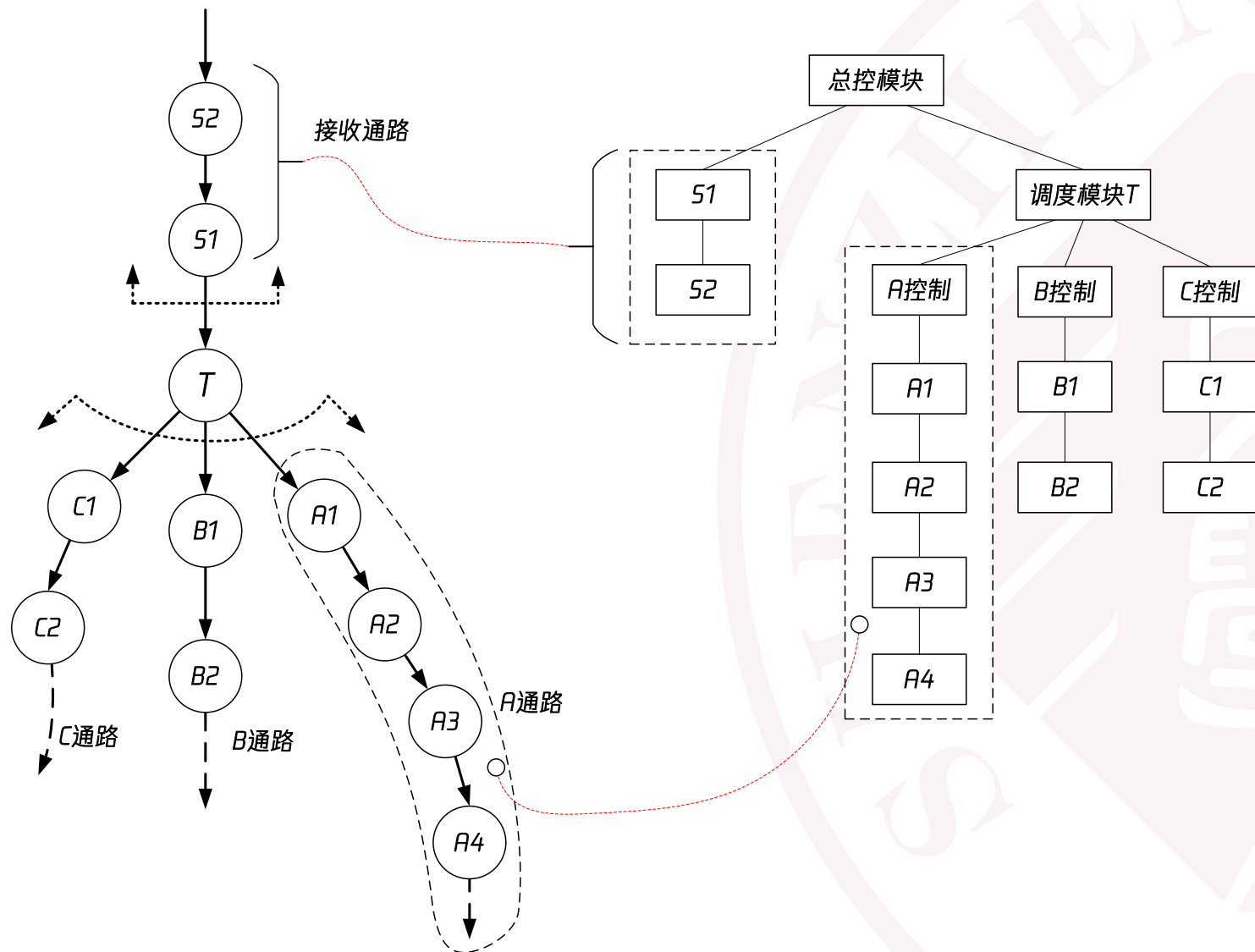
② 事务流

系统根据状态或者处理将业务进行分类处理的数据流。



事务流由至少一条输入路径、一个事务处理中心和若干条活动通路组成。

3.系统结构设计



4.数据设计

软件系统中的数据

- 模块之间交换的数据
- 系统处理过程中需要的数据
- 系统存储的数据

4.数据设计

[1] 数据元素设计



4.数据设计

固定聚合类型的成分由确定数目的成分按照特定结构组成;
可变聚合类型的成分构成不确定。



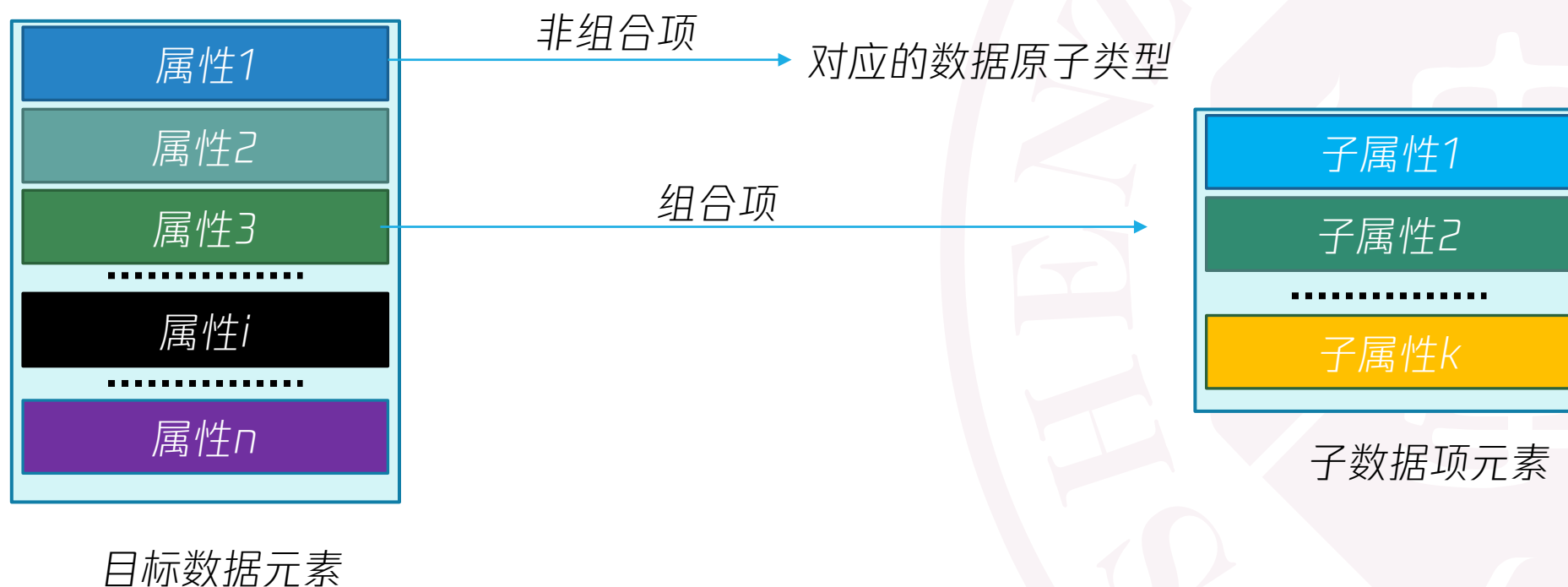
固定聚合类型



可变聚合类型

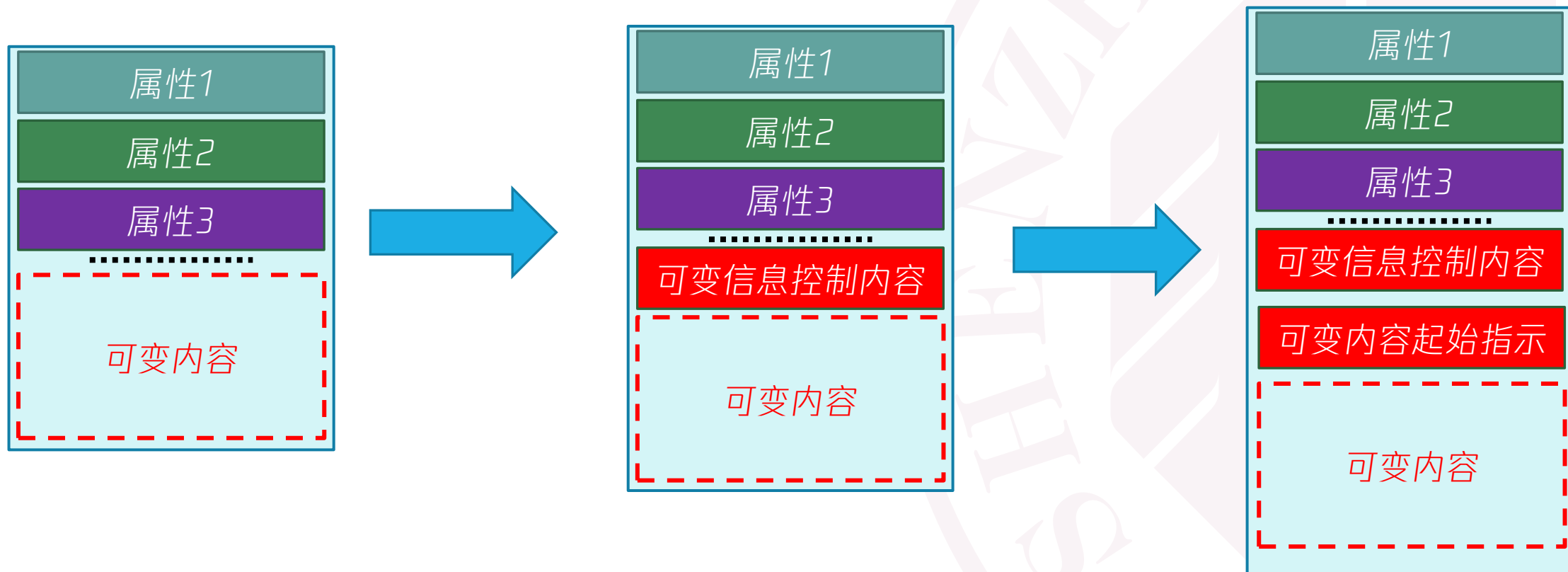
4.数据设计

固定聚合类型映射关系



4.数据设计

可变聚合类型映射关系

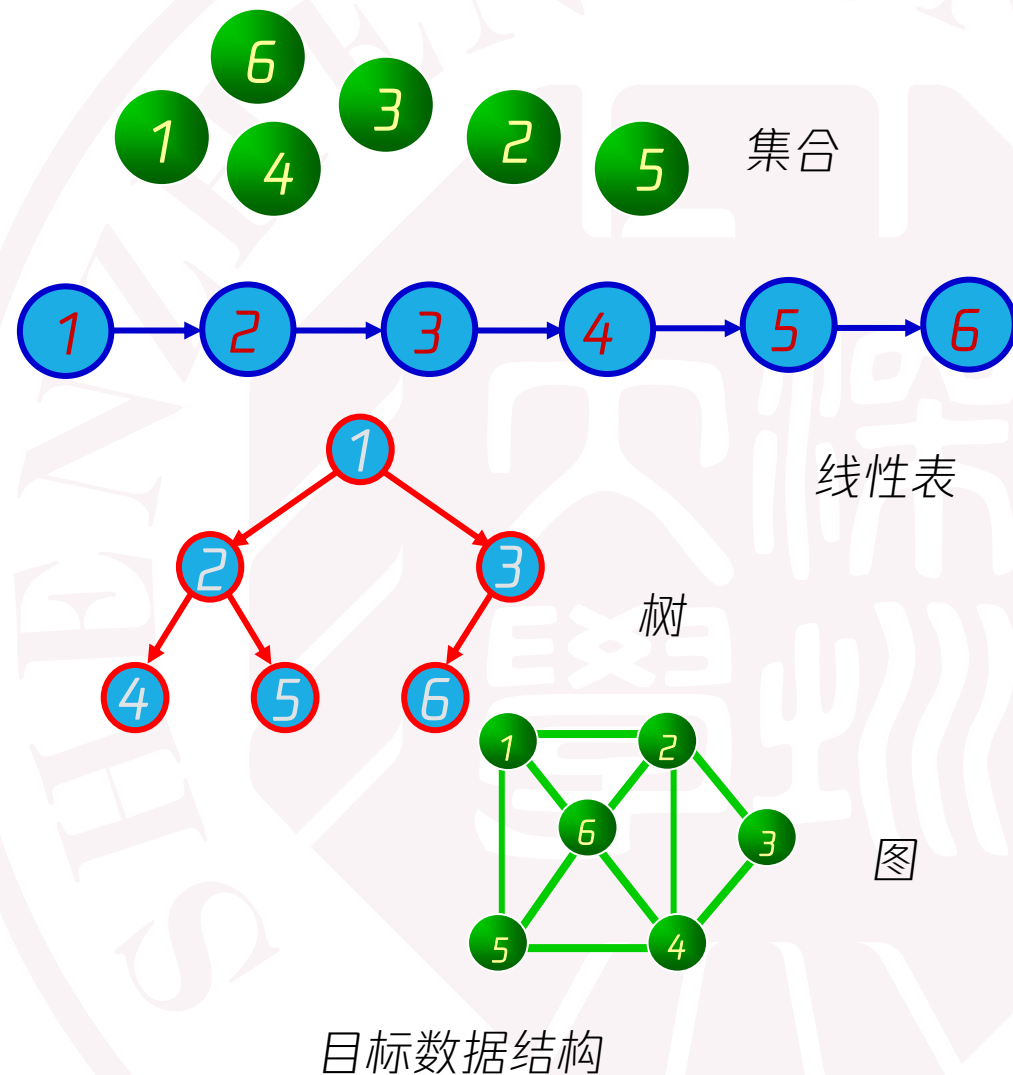


4.数据设计

[2] 数据结构设计



数据模型



4.数据设计

[3] 文件格式设计

如果涉及到数据的持久化，即保存数据或者读取数据，就需要考虑将程序运行的输入和输出信息保存为数据文件。

文件：存储在计算机外部介质上的数据的集合。

二进制文件

ASCII文件

4.数据设计

根据数据类型设计文件存储方式

① 固定聚合类型〔数据元素的内容和长度固定〕

以数据元素为单位，依次将多条数据写入到同一个文件中，使用一个文件保存多条数据记录。

② 可变聚合类型〔数据元素的长度不固定〕

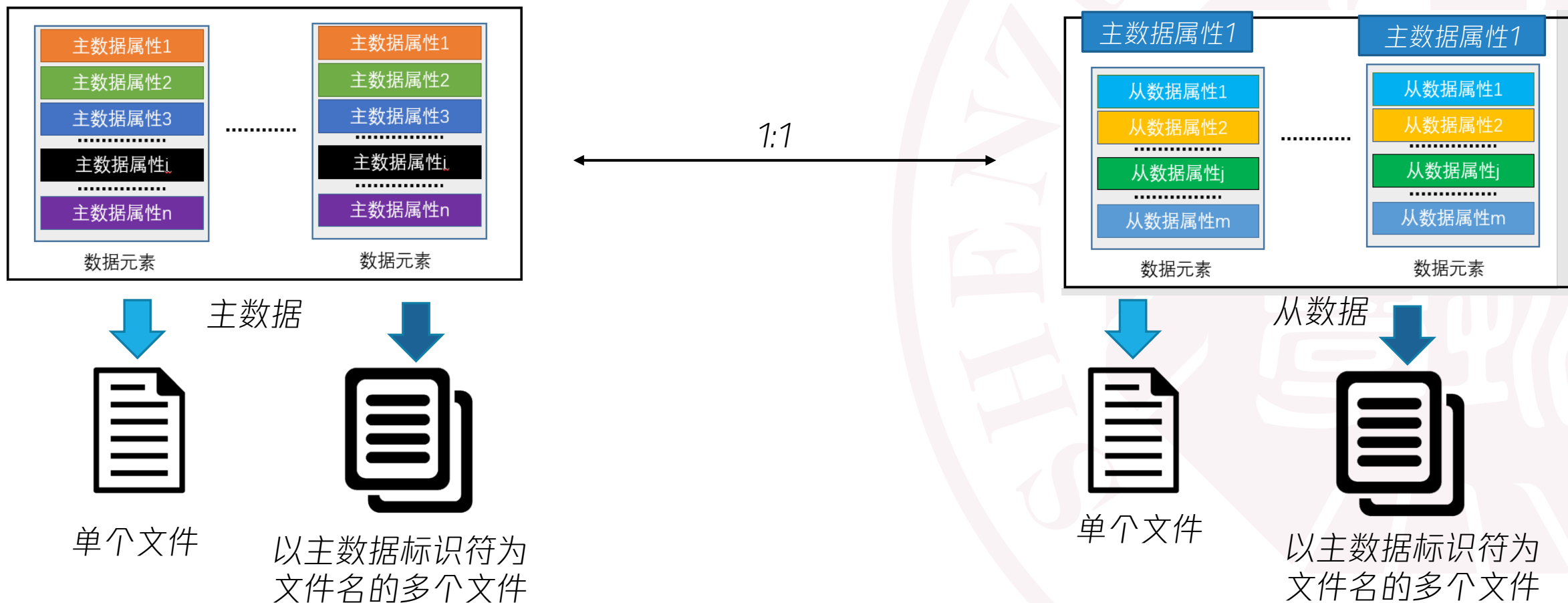
将每条数据元素单独保存为一个文件，以文件为单位对多个数据元素进行存储。

使用文件的名称来标识不同的数据记录。

4.数据设计

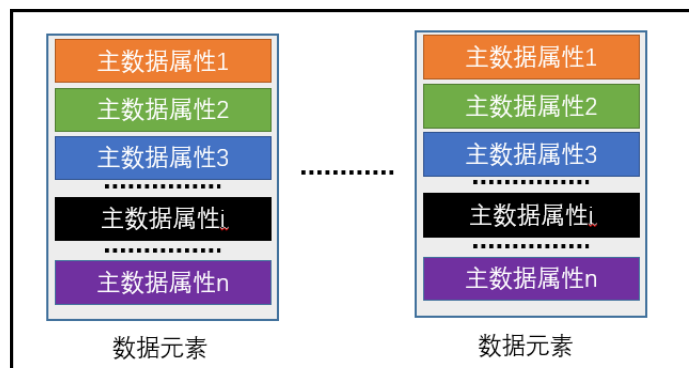
根据数据关系设计文件存储关系

① 一对一

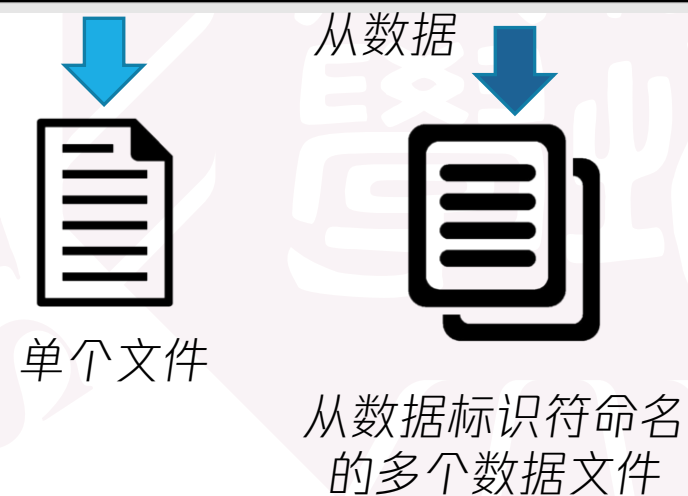
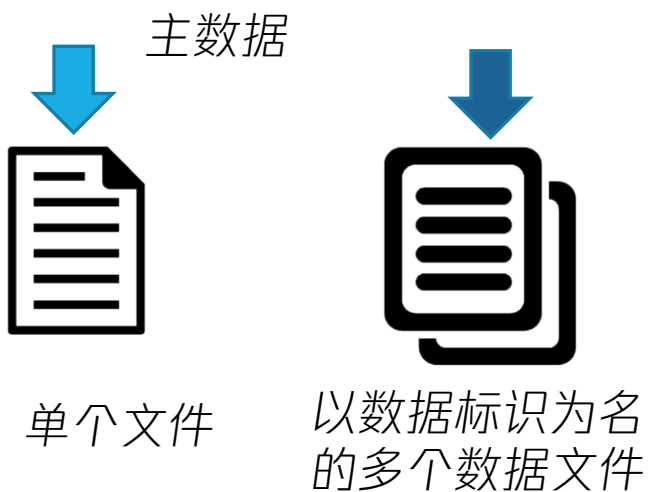
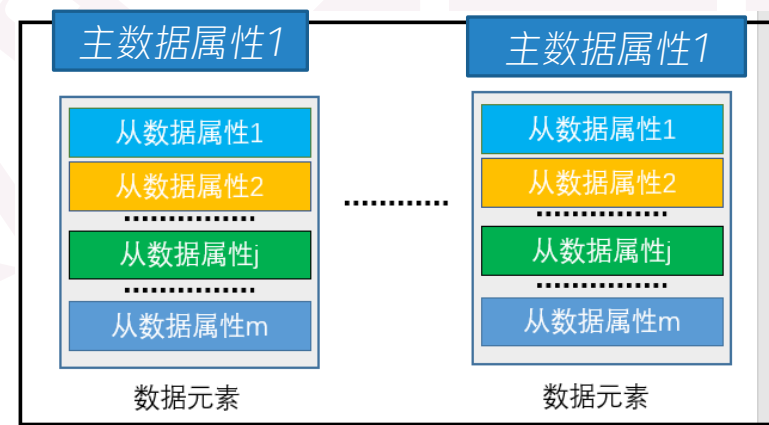


4.数据设计

②一对多

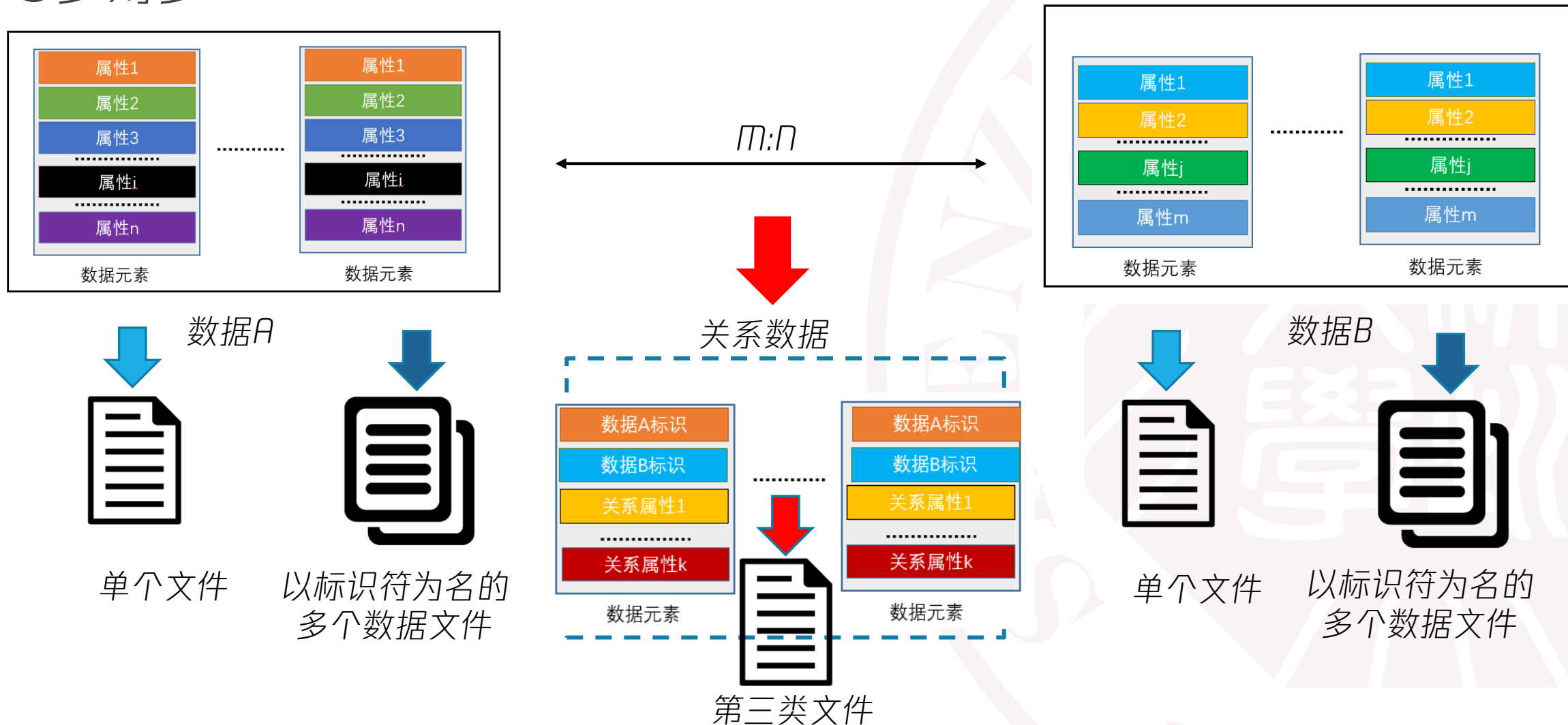


1:n



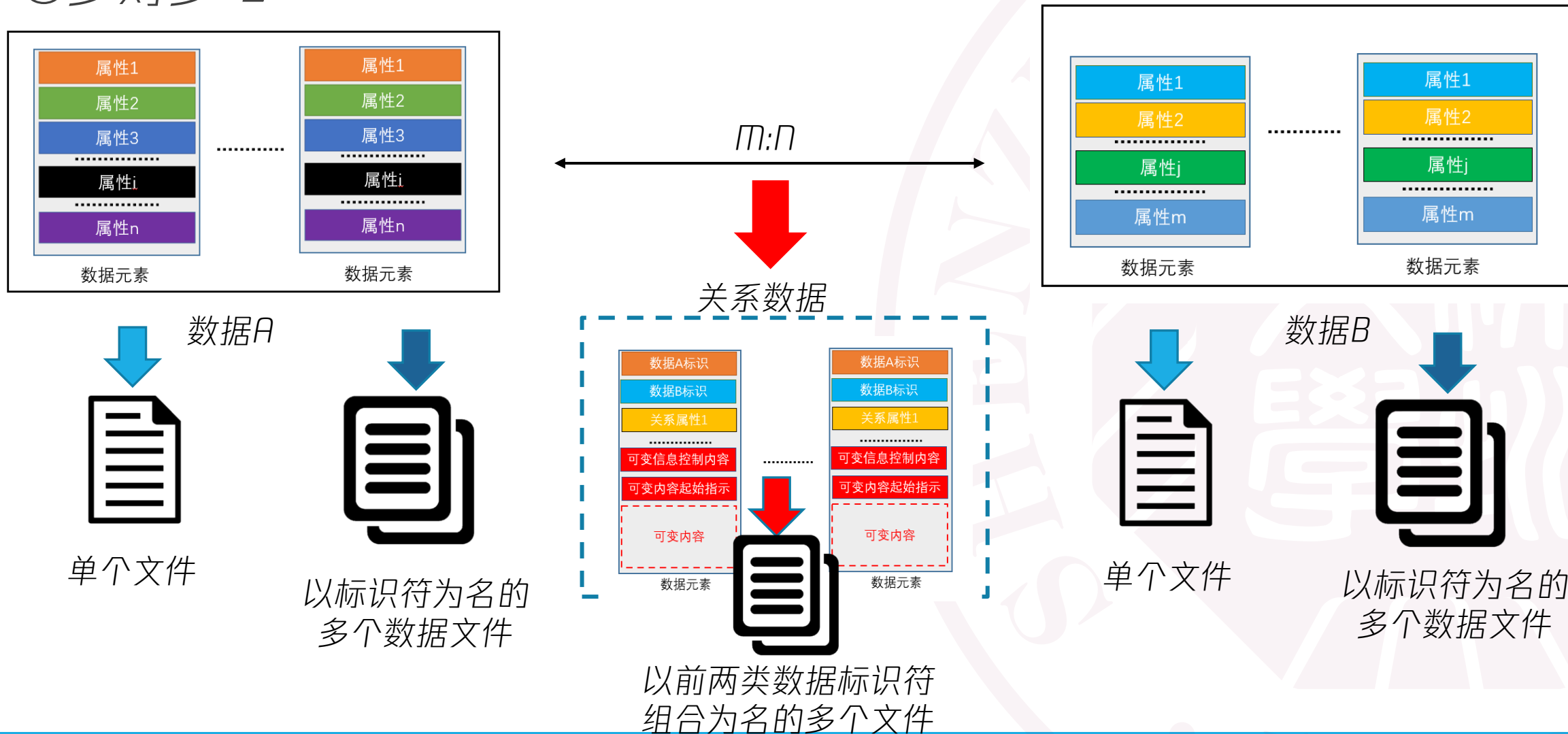
4.数据设计

③多对多-1



4.数据设计

③多对多-2



4.数据设计

[4] 数据库设计

结合给定的应用需求，构造〔设计〕优化的数据库逻辑模型和物理模型，并建立数据库及其应用系统。

在设计数据库之前，软件设计人员首先需要确定各个数据元素的标识符，即主键。

根据数据库表之间关系设计数据库

一对一

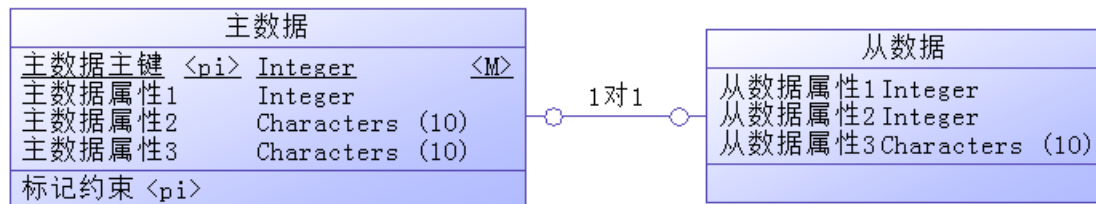
一对多

多对多

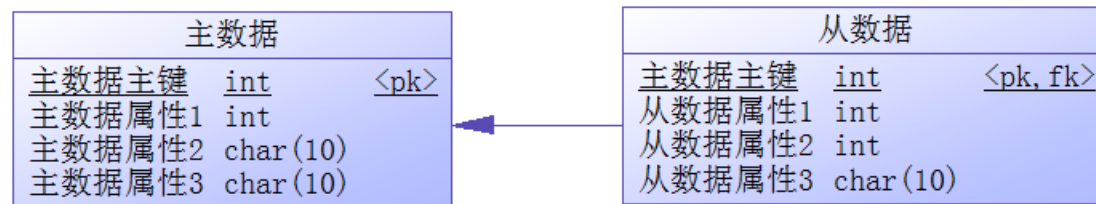
4.数据设计

[4] 数据库设计

① 一对一



逻辑模型

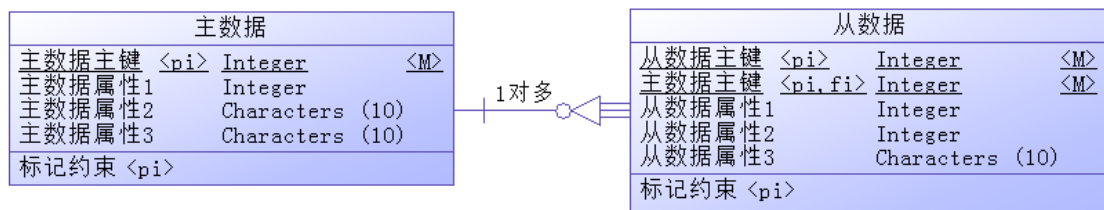


物理模型

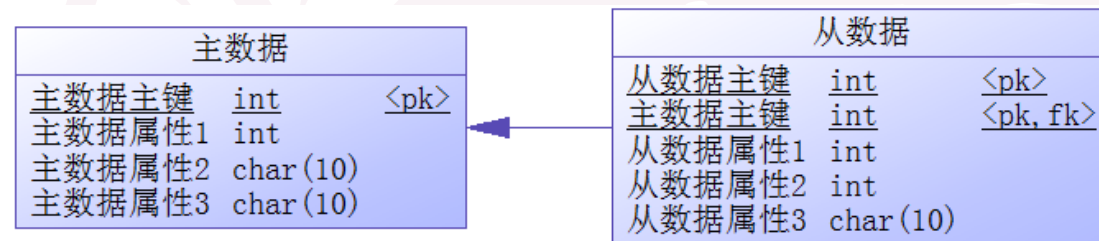
4.数据设计

[4] 数据库设计

② 一对多



逻辑模型

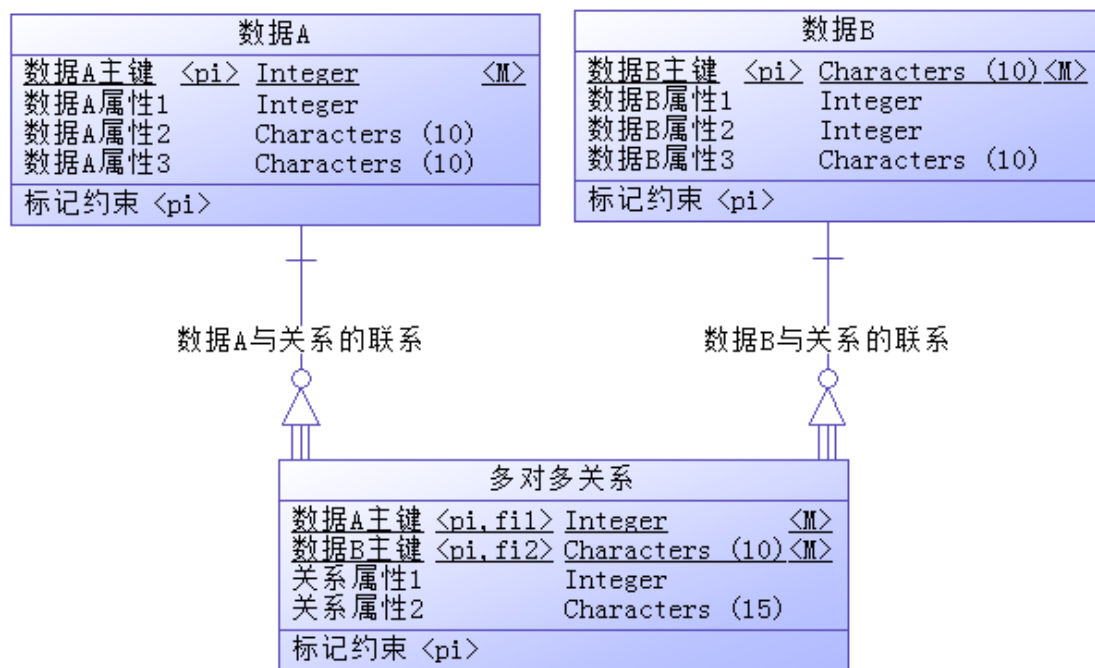


物理模型

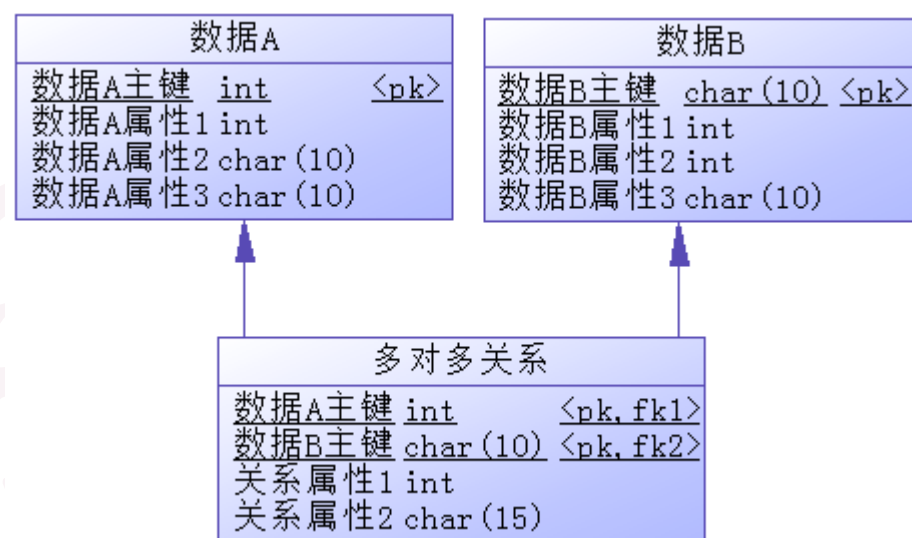
4.数据设计

[4] 数据库设计

③ 多对多



逻辑模型



物理模型

4.数据设计

数据库物理模型评价

时间效率

- 对数据库系统的存取方式进行评估
- 使用技术：索引，聚簇等

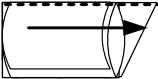
空间效率

- 对数据的存储结构进行度量
- 使用技术：规范化理论

5. 过程设计

对各个模块使用的算法或者数据结构进行设计，并采用合适的建模工具对模块内容进行准确描述。

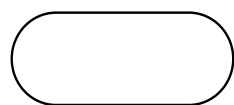




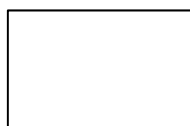
5. 过程设计

[1] 程序流程图

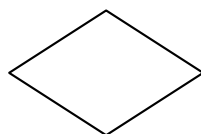
又称程序框图，采用统一的符号来描述处理流程，具有结构清晰、逻辑性强等特点，能够直观地描述程序的控制处理逻辑。



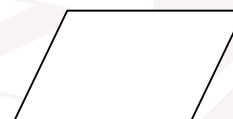
开始/结束



流程



判定



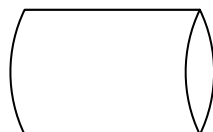
数据



文档



子流程



数据库



外部数据

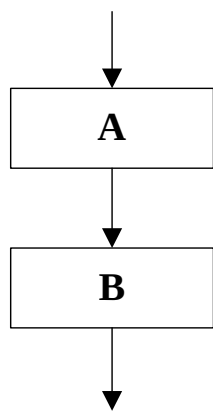


程序流向

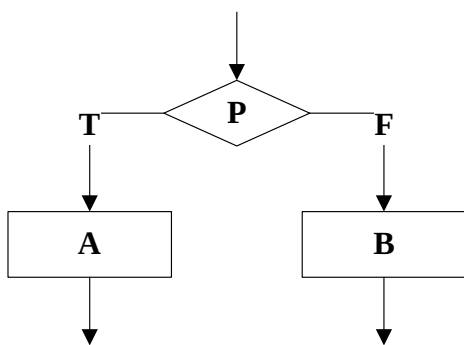


注释

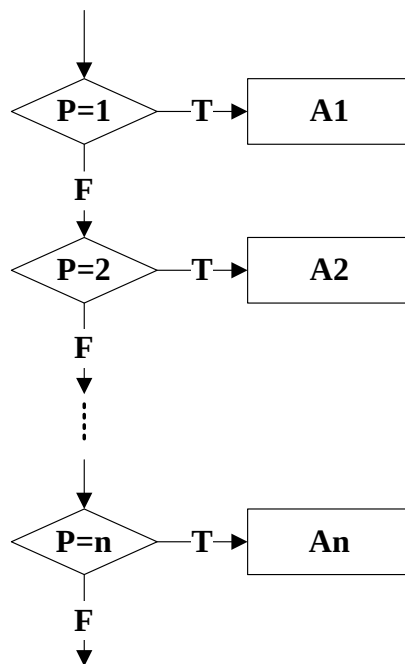
5. 过程设计



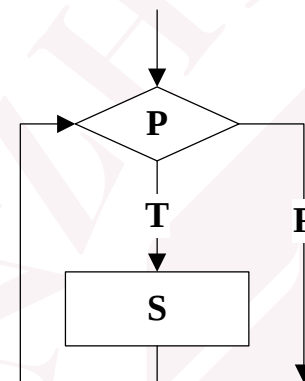
顺序结构



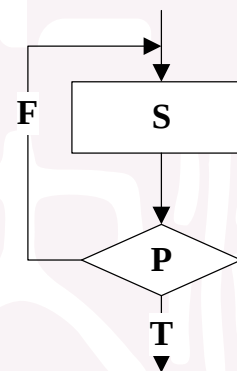
选择结构



多分支



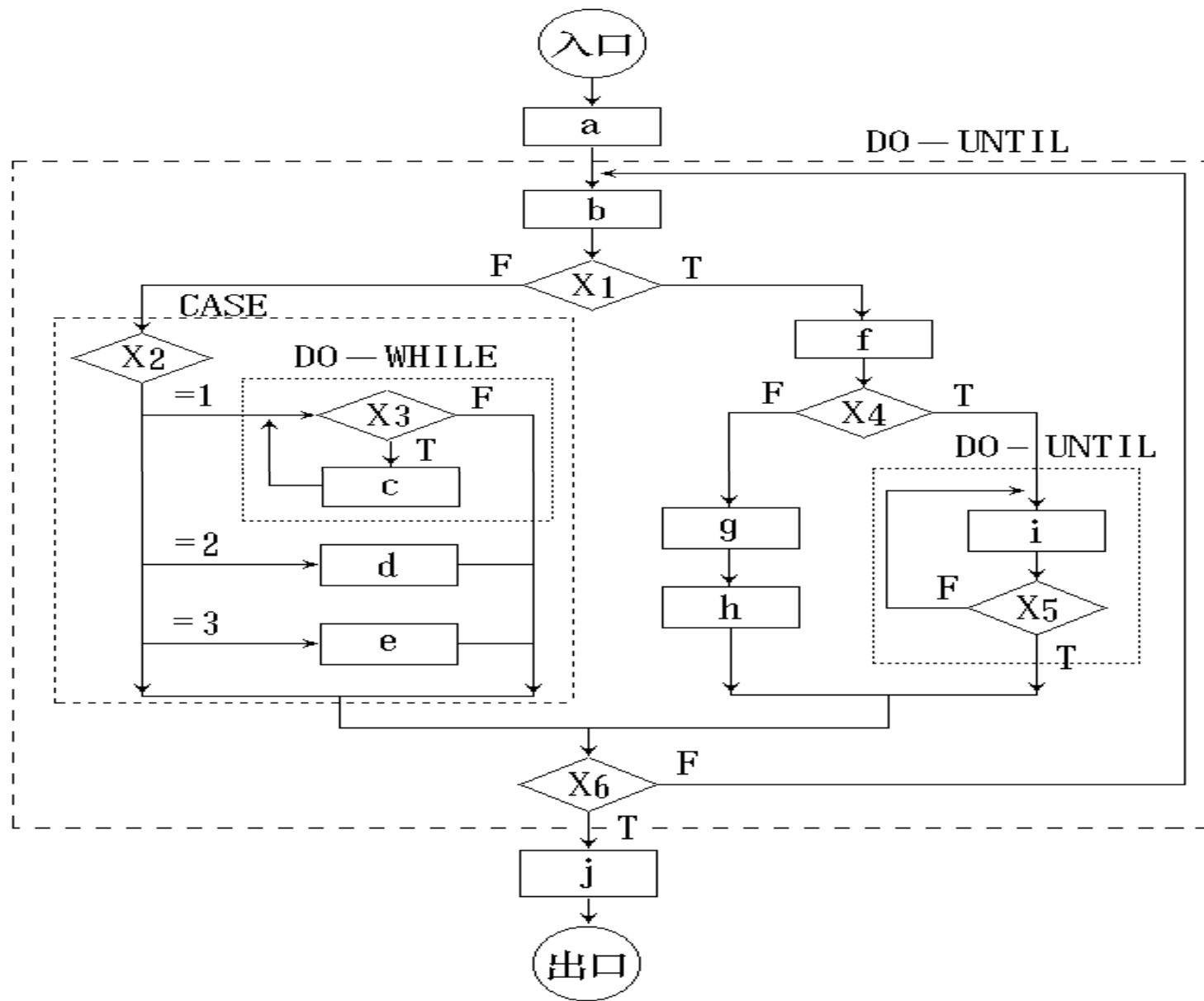
先判定型循环



后判定型循环

循环结构

5. 过程设计



5. 过程设计

程序流程图的主要缺点:

- 程序流程图本质上不是逐步求精的好工具

它诱使程序员过早地考虑程序的控制流程，而不考虑程序的全局结构。

- 程序流程图中用箭头代表控制流

程序员不受任何约束，可以完全不顾结构程序设计的精神，随意转移控制。

- 程序流程图不易表示数据结构

- 详细的微观程序流程图——每个符号对应于源程序的一行代码

对于提高大型系统的可理解性作用甚微。

5. 过程设计

[2] 过程设计语言(PDL, Program Design Language)

Algorithm 1 Federated Averaging Algorithm

```
1: Input: Number of global iterations  $T$ 
2: Input: Number of local training epoches  $E$ 
3: Input: Desired level  $A_T$  of model accuracy
4: Output: Final parameters  $\theta_t$ 
1: procedure FEDAVG
2:   ServerGlobalUpdate:
3:     Initialize parameters  $\theta_0$ 
4:     Repeat
5:        $C \leftarrow$  Select a subset of clients
6:       for each client  $c \in C$  do
7:          $\theta_t^c \leftarrow$  ClientLocalUpdate( $\theta_{t-1}$ )
8:       end for
9:        $\theta_t \leftarrow \sum_C \frac{n^c}{n} \theta_t^c$ 
10:      Until  $A_T$  is attained
11:      ClientLocalUpdate( $\theta$ )
12:      for local iteration  $e = 1$  to  $E$  do
13:        for each each batch  $b$  in client's split do
14:           $\theta = \theta - \eta \Delta L(b; \theta)$ 
15:        end for
16:      end for
17:      return local model  $\theta$ 
18: end procedure
```

6. 设计优化

设计启发式规则 [1] :

- ✓ 在有效的模块化机制前提下，首先使用最少量的模块来实现系统功能;
- ✓ 在满足信息要求的前提下，尽量使用最简单的数据结构;
- ✓ 在优化设计内容时，优先设计和优化对时间起决定性作用的模块;
- ✓ 首先在不考虑时间因素的前提下开发并精化软件结构;

6. 设计优化

设计启发式规则 [2] :

- ✓ 评估并优化模块的处理过程，降低模块处理的时间复杂度和空间复杂度；
- ✓ 在保证模块功能的情况下，简化条件表达式，优化逻辑处理过程；
- ✓ 尽量使用高级程序设计语言来编写程序；
- ✓ 定位并孤立出软件中占用大量处理器资源的模块；必要时，重新设计或者采用依赖于机器的语言重写上述模块的代码。

7. 小结

结构化设计从**功能**角度出发，将目标软件分解为多个独立的功能模块；

- ◆优秀的软件模块内聚程度较高，且与其他模块耦合松散；
- ◆可以根据变换流和事务流的特点将数据流图映射为系统结构图；
- ◆IPO表对各个模块的输入数据、输出数据和内部数据进行说明；
- ◆数据模型是软件设计中的重要环节〔数据元素设计、文件设计、数据库设计等〕；
- ◆将处理对应的“处理/加工逻辑说明”和“状态转换图”转换为具体处理流程；
- ◆软件设计人员可以结合已有的软件设计经验和启发式规则来优化软件设计。

谢谢大家!

课后思考:

1. 了解了结构化设计后，你觉得怎样才能做好结构化分析？
2. 结构化方法的本质是什么？
3. 结构化设计需要细致到什么程度才是合适的？